MRI PROJECT · GUIDE 2 OF 2

Running MMedPO on the HGX Cluster

From zero to timed inference: what a GPU cluster is, how Slurm works, every script we used to run the model on hgx-01, what we measured, and a hands-on lab so you can do it yourself.



hgx-01 · 8 × NVIDIA B200

Workspace: ~/MRI-MMedPO on aihub-cluster1 · Measurements from Slurm jobs 263–274, 28–29 October 2026

Read Guide 1 (the paper) first. This guide only reminds you of the AI ideas it needs.

Contents

How to read this guide ... 3

PART I · CONCEPTS YOU NEED

What "running a model" means ... 5
The hardware: CPUs, GPUs and memory ... 8
Anatomy of our cluster ... 11
Slurm from zero ... 13
GPUs on this cluster: the "no GRES" quirk ... 17

PART II · THE SOFTWARE STACK

Python, packages and virtual environments ... 20
PyTorch, CUDA and why the B200 needed newer versions ... 23
Hugging Face: where the model and data come from ... 26

PART III · OUR PIPELINE, STEP BY STEP

The map: folders, files and the six steps ... 29
Step 1 · Build the environment and download everything ... 31
Step 2 · Turn the test sets into question files ... 34
Step 3 · Checkpoints: base model plus MMedPO adapter ... 36
Step 4 · The timing script, line by line ... 39
Step 5 · Four runs at once ... 44
Step 6 · Collecting the numbers ... 46

PART IV · RESULTS

What we measured ... 48
What these numbers do and do not tell you ... 53

PART V · DO IT YOURSELF

Hands-on lab ... 55
When things go wrong ... 61
What training would take ... 63

APPENDICES

A · Command cheat sheet ... 64
B · Glossary ... 65

How to read this guide

Your advisor asked two questions: *can the MMedPO model run on our cluster?* and, if it can, *how long does it take to answer?* The short answers are **yes, easily**, and **about 0.1–0.2 seconds per question and 0.7–1.0 seconds per X-ray report**. This guide explains where those answers came from, from the ground up.

Guide 1 explained what MMedPO *is*: a medical vision-language model (LLaVA-Med 1.5, about 7 billion numbers) that was fine-tuned with a clinically weighted version of preference optimisation (DPO). This guide is about what it takes to *run* that model on real hardware. We assume you have never run a model yourself and have only a rough idea of what a cluster is.

> **THE WHOLE GUIDE IN FIVE SENTENCES**
>
> A trained model is a big file of numbers. "Running" it (inference) means loading those numbers into a GPU's memory and pushing your image and question through them to get an answer back, one word-piece at a time. Our cluster has one GPU computer, `hgx-01`, with eight NVIDIA B200 GPUs; you never use it directly but ask a scheduler called Slurm to run your commands there. We built a Python environment with a PyTorch version new enough for B200s, downloaded the model and four test sets, and timed the model on every test question with a small script. The model needs about 17 GB of the B200's 183,359 MiB of memory and answers a question in about 0.1–0.2 s and writes a report in about 0.7–1.0 s.

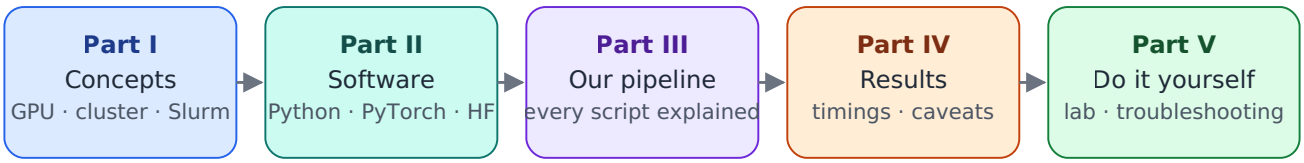


Figure 1. The five parts of this guide. Each builds on the one before; Part III is the heart of the guide, and Part V is where you type commands.

Conventions

- Text in `this font` is something you type, a file name, or program output. Dark boxes are terminal sessions. A green `$` starts a line you type; the lines below it are what the computer printed back. **Every terminal output in this guide is real.** It was copied from our runs on 28–29 October 2026, sometimes shortened (shown with `…`).
- File names like `scripts/run_bench.sbatch` are relative to the project folder `~/MRI-MMedPO`. The `~` means "my home folder", `/home/shovan.1`.
- Coloured boxes: Key idea the one thing to remember, Analogy an everyday comparison, Watch out a trap we hit or you could hit, Try it something to type, Recap the end-of-chapter summary.

Part I

Concepts you need

What it means to run a model, what a GPU is and why we need one, what a cluster is, and how to ask Slurm, the cluster's scheduler, for a GPU.

CHAPTER 1

What “running a model” means

Before touching the cluster, it helps to have a clear picture of what physically happens when “the model answers a question”. This chapter builds that picture. Everything in the rest of the guide hangs on it.

1.1 A model is a file of numbers plus the code that uses them

Guide 1 described LLaVA-Med as a network with about 7 billion **parameters** (also called **weights**). Concretely, those weights are stored on disk as ordinary files. Here is the base model folder on our cluster. The four `.safetensors` files *are* the model: about 15.1 GB of numbers in total. (safetensors is a standard file format for big arrays of numbers; large models are split into several such *shards*.)

```
$ ls -la checkpoints/llava-med-v1.5-mistral-7b/
-rw-r--r-- 1 shovan.1 shovan.1       1407 Oct 28 22:14 config.json
-rw-r--r-- 1 shovan.1 shovan.1 4943162336 Oct 28 22:14 model-00001-of-00004.safetensors
-rw-r--r-- 1 shovan.1 shovan.1 4999819336 Oct 28 22:14 model-00002-of-00004.safetensors
-rw-r--r-- 1 shovan.1 shovan.1 4927408360 Oct 28 22:14 model-00003-of-00004.safetensors
-rw-r--r-- 1 shovan.1 shovan.1  262144128 Oct 28 22:14 model-00004-of-00004.safetensors
-rw-r--r-- 1 shovan.1 shovan.1     493443 Oct 28 22:14 tokenizer.model
…
```

The numbers alone do nothing. You also need **code** that knows the network’s shape (which numbers multiply which) and runs the arithmetic. For MMedPO that code is the LLaVA library inside the authors’ GitHub repository (`MMedPO/train/dpo/llava/`). It is built on two large open-source libraries: **PyTorch**, which does the maths on the GPU, and Hugging Face **transformers**, which provides standard model building blocks. `config.json` tells the code which shape to build. `tokenizer.model` is the dictionary that turns text into tokens and back (Guide 1, “tokens”).

1.2 Training versus inference

There are two completely different things you can do with a model, and the advisor’s question was about only one of them.

- **Training** (also called fine-tuning, when it starts from an existing model) changes the weights. You show the model examples, measure how wrong it is (the *loss*; for MMedPO the weighted DPO loss of Guide 1), and nudge every weight slightly to reduce it. This is repeated thousands of times. It needs a lot of memory (the model, plus the gradients and optimiser state used for the nudging) and hours of GPU time.
- **Inference** means using the finished weights to answer. Nothing is learned and the weights are *frozen*. This is what a hospital would do with the model, and it is what your advisor asked about: “we are interested to know the inference time, not the training time.”

The MMedPO authors released their trained weights on Hugging Face, so we never had to train anything. We downloaded their result and ran inference.

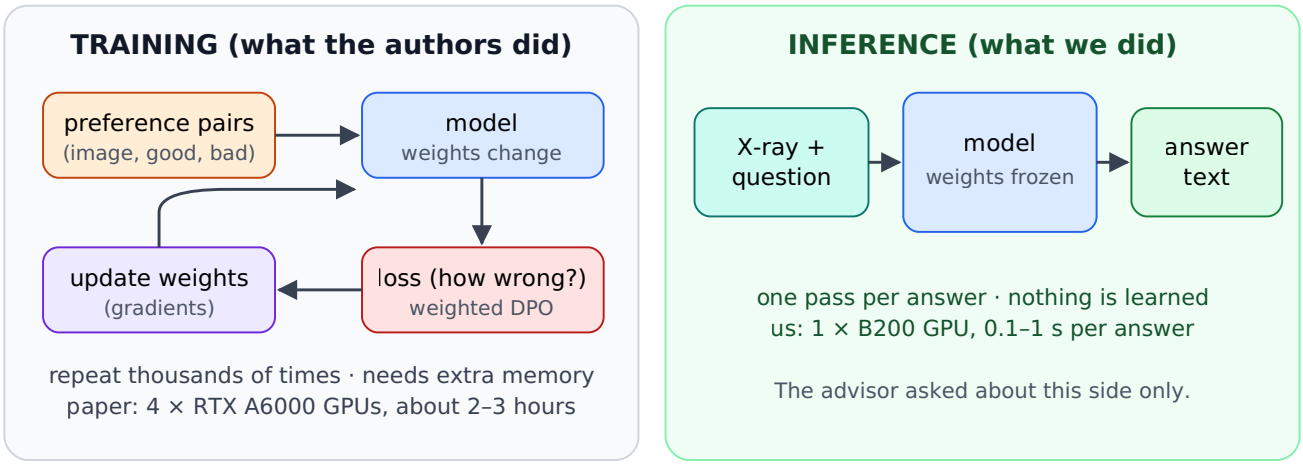


Figure 2. Training adjusts the weights in a loop. Inference uses fixed weights to produce answers. MMedPO's contribution is a training method, but its released result is a set of weights you can run with ordinary inference.

1.3 What happens when the model answers one question

Let us follow a single request through the machine. The example is real: question 0 of the VQA-RAD test set, *"is there evidence of an aortic aneurysm?"*, asked about a chest X-ray.

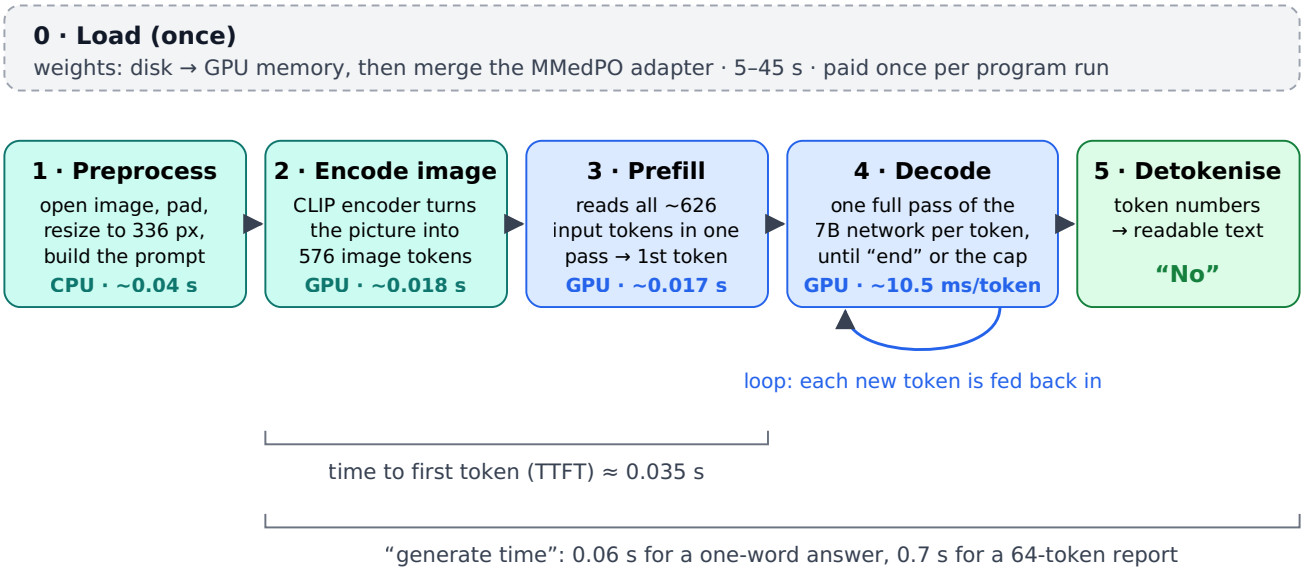


Figure 3. The life of one request, with average times we measured on one B200 (Part IV has the details). The decode loop (step 4) is where almost all the time goes for anything longer than a one-word answer. The 0.017 s prefill is TTFT minus the image encoding. Step 1 runs on the CPU, steps 2–4 on the GPU.

1. **Load (once).** The weights are copied from disk into the GPU's own memory and the MMedPO adapter is merged in (Chapter 12). This takes seconds to tens of seconds, but is paid only once per program run. After that the model can answer any number of questions.

2. **Preprocess.** On the ordinary processor (CPU), the program opens the image file, pads it to a square, shrinks it to 336 × 336 pixels, and converts the question into numbers (tokens) inside a fixed conversation template (Chapter 13 shows the exact text the model sees).
3. **Encode the image.** The CLIP vision encoder cuts the image into 24 × 24 = 576 patches and turns each into a vector. A small "projector" network converts those vectors into the language model's format. The image is now 576 "image tokens" sitting in front of the text tokens.
4. **Prefill.** The language model (Mistral-7B inside LLaVA-Med) reads all ~626 input tokens in one big parallel pass and predicts the first output token.
5. **Decode.** Each further token needs another full pass through the network, because each new word depends on the words before it. This is a *loop*, and it cannot be parallelised for a single answer. It stops when the model produces a special "end" token or hits a length cap (`max_new_tokens`).
6. **Detokenise.** The token numbers are turned back into text: "No".

> **ANALOGY**
> Prefill is a student *reading* the exam question: they take in the whole page at once, and it is quick. Decode is the student *writing* the answer, one word after another, re-reading what they wrote before each new word. A one-word answer is fast. A paragraph-long report takes about ten times longer. Our measurements show exactly this: reports are 5–8× slower per sample than questions.

1.4 Words we will use for speed

Term	Meaning	Our typical value
Latency	How long one request takes, start to finish	0.12–0.19 s question; 0.7–1.0 s report
TTFT	"Time to first token": steps 2 and 3	≈ 0.035 s
Decode speed	Tokens per second during step 4	94–124 tokens/s
Throughput	Requests finished per second overall	1.0–8.1 per second
Load time	One-time cost of step 0	5–45 s

Table 1. The speed vocabulary used throughout the guide. Numbers come from Part IV.

> **RECAP**
> - A model = weight files (15.1 GB here) + code that uses them (LLaVA on PyTorch).
> - Training changes weights; inference uses frozen weights. We measured inference only.
> - One answer: preprocess → encode image → prefill → decode loop → detokenise.

CHAPTER 2

The hardware: CPUs, GPUs and memory

The first part of the advisor's question, "is it possible to run in the cluster, or is there a problem such as memory?", is really a question about hardware. This chapter gives you the two facts that decide it: what a GPU is good at, and how much memory the model needs.

2.1 CPU and GPU: a few experts versus a stadium of calculators

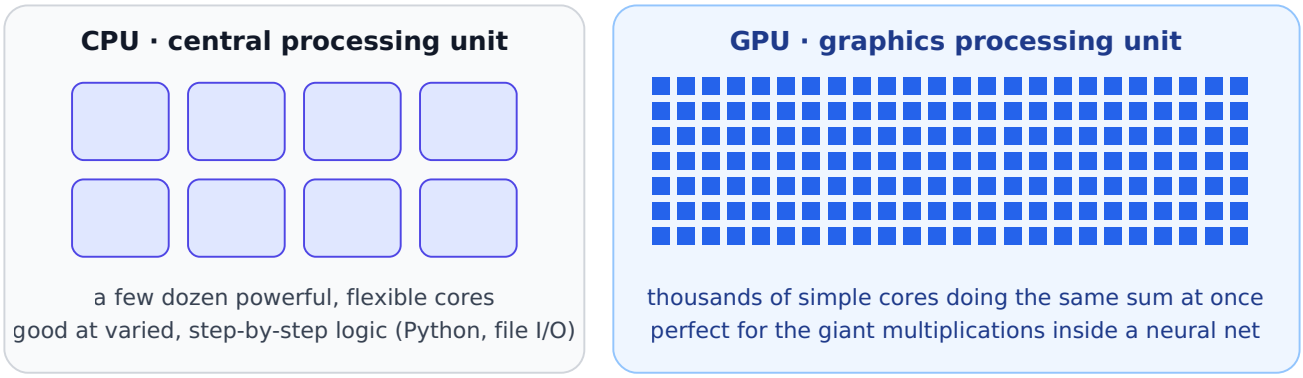


Figure 4. A CPU has a few flexible cores. A GPU has thousands of simple ones that all perform the same arithmetic on different numbers at the same time. The drawing is schematic, not to scale.

Almost all of the work inside a neural network is **matrix multiplication**: multiplying large tables of numbers together. Every one of the billions of multiply-and-add operations is independent of the others, so they can all happen at once. GPUs were invented to colour millions of screen pixels at the same time, which is the same kind of work, and they turn out to be ideal for neural networks. Running a 7-billion-parameter model on CPUs alone is possible but painfully slow. On a modern GPU it is fast.

Our GPUs are **NVIDIA B200**s, NVIDIA's current-generation data-centre GPU, from the architecture family called **Blackwell**. That name matters later, in Chapter 7.

2.2 Memory: why it decides "can it run at all?"

A GPU has its own memory, separate from the computer's main memory (RAM). It is often called **VRAM** or "GPU memory". To run a model, *all of its weights must fit in GPU memory*, plus some working space. If they do not fit, the program stops with an "out of memory" error. That is the problem your advisor was asking about.

The memory needed is easy to estimate: *number of parameters* × *bytes per parameter*. Each number can be stored at different precisions:

Precision	Bytes per number	7.57 billion numbers need	Used by us?
fp32 (“full precision”)	4	≈ 30 GB	no
fp16 / bf16 (“half precision”)	**2**	**≈ 15 GB**	**yes: fp16 for inference**
int8 / int4 (“quantised”)	1 / 0.5	≈ 7.6 / 3.8 GB	no (would change answers slightly)

Table 2. Memory for the weights alone. LLaVA-Med v1.5 has about 7.57 billion parameters, which is what you get by dividing the 15.13 GB of weight files by 2 bytes each. That covers the Mistral-7B language model, the CLIP vision encoder and the projector.

On top of the weights, inference needs a little working memory. It holds the intermediate results of each layer and, importantly, the **KV cache**: the model’s saved notes about every token so far, which let it avoid re-reading the whole prompt for every new token. This grows with the length of the input plus the output. With ~626 input tokens and at most a few hundred output tokens, it stays small.

Here is what we actually measured with PyTorch’s own memory counters (`torch.cuda.memory_allocated` and `max_memory_allocated`):

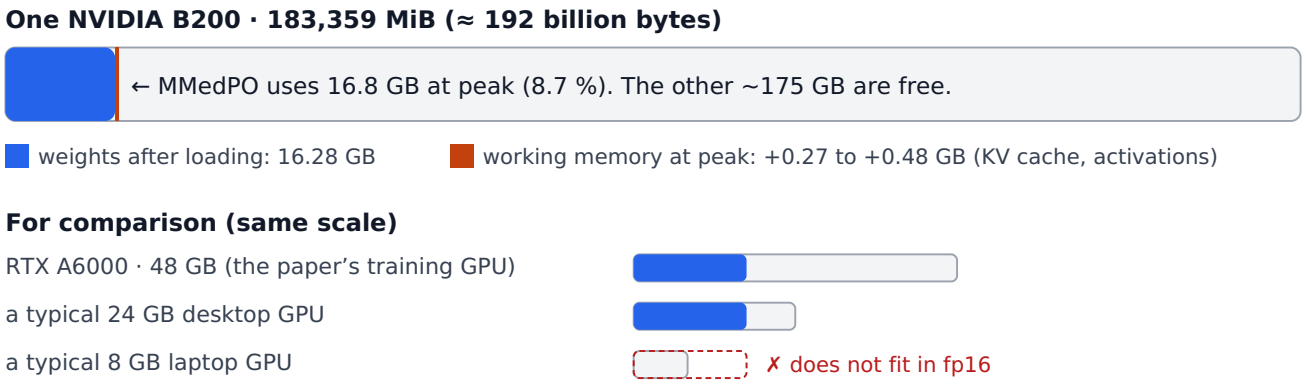


Figure 5. Memory use measured in all four of our runs (16.56–16.76 GB peak), drawn to scale against one B200. The comparison bars are for intuition only. We did not run on those GPUs.

> **KEY IDEA · THE ANSWER TO “IS MEMORY A PROBLEM?”**
> No. The model needs about 17 GB and one B200 has 183,359 MiB. Memory is not remotely a constraint: you could fit about ten copies of the model on a single B200. The real constraint on this cluster is *time* (the 6-hour job limit) and *sharing* (other people use the same GPUs), not memory.

WATCH OUT · GB, GIB AND MIB

Memory is quoted in two systems. **GB** (gigabyte) = 10^9 bytes. **GiB** (gibibyte) = 2^{30} bytes ≈ 1.074 GB. `nvidia-smi` reports MiB (2^{20} bytes), so a B200 shows as "183359 MiB", which is ≈ 179 GiB ≈ 192 GB. Our timing script reports GB (10^9). We sometimes say "a 183 GB GPU" loosely. The exact figure is 183,359 MiB.

2.3 The rest of the machine: CPUs and RAM still matter

The GPU does the heavy maths, but the CPU still has a job. It runs Python, reads image files from disk, resizes them, and feeds the GPU. The computer's main memory (RAM) is where the weights pass through on their way from disk to the GPU. Our compute node has plenty of both: 224 CPU cores available to jobs and about 2.3 TB of RAM (Chapter 3). One detail matters: on this cluster *you only get RAM in proportion to the CPU cores you ask for* (Chapter 4).

RECAP

- GPUs do huge numbers of identical multiplications in parallel, which is exactly what a neural network needs.
- Rule of thumb: memory ≈ parameters × 2 bytes in fp16. For LLaVA-Med that is ≈ 15 GB, and we measured 16.8 GB peak.
- One B200 (183,359 MiB) holds the model ten times over. Memory is not a problem.

CHAPTER 3

Anatomy of our cluster

A “cluster” sounds grand, but ours is easy to picture: one computer you log into, one big GPU computer that does the work, a shared disk both of them can see, and a piece of software that decides who gets to use the GPU computer when. Think of a restaurant: the login node is the *lobby* where everyone waits and orders, hgx-01 is the *kitchen* with eight top-of-the-line stoves (the GPUs), and Slurm is the *host* who finds you a free stove for at most six hours.

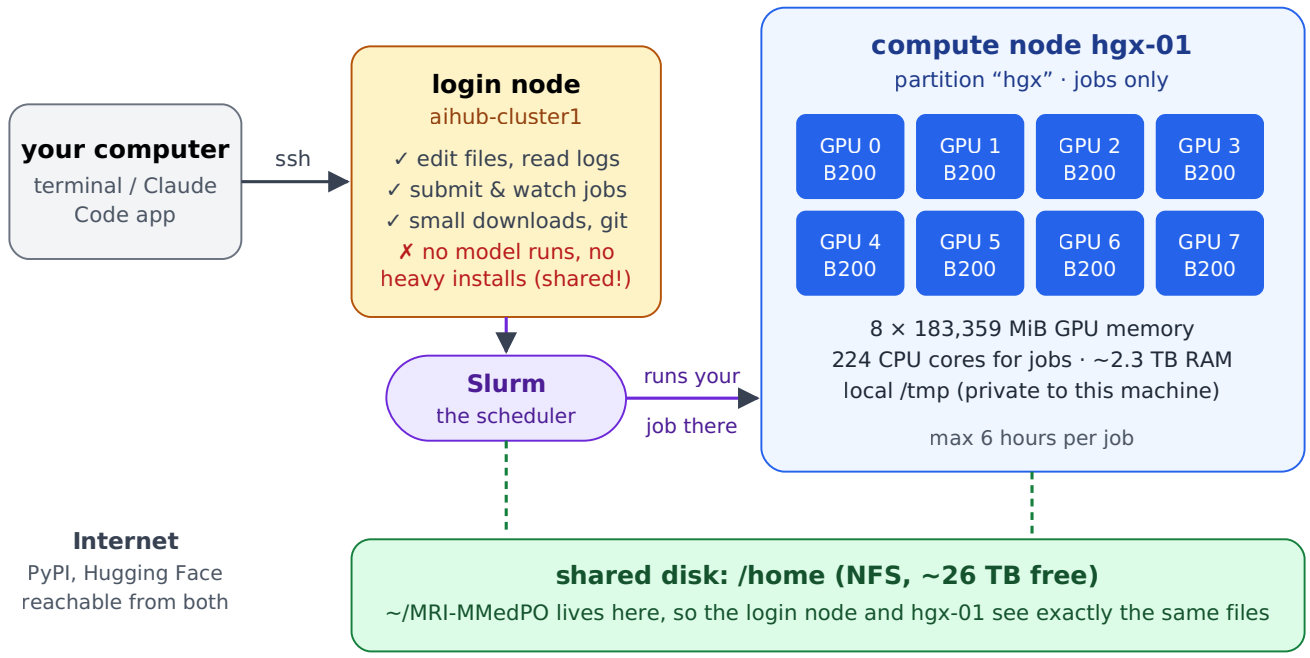


Figure 6. The pieces of our cluster and how they connect. You type commands on the login node. Slurm runs your heavy work on hgx-01. Both machines see the same `/home` disk.

3.1 The login node: the front door

When you connect to the cluster (with `ssh`, or through the Claude Code app’s terminal) you land on the **login node**, a machine called `aihub-cluster1`. Everyone who uses the cluster lands here. It is meant for light work: editing files, looking at results, submitting jobs and checking on them. Running a 7-billion-parameter model here would slow everyone else down, and it has no B200s anyway. That is why your instruction for this project was “ensure to get interactive session/Slurm jobs, not login node”.

3.2 The compute node: where the work happens

The real hardware is one large computer, **hgx-01** ("HGX" is NVIDIA's name for its 8-GPU server boards). It has eight B200 GPUs, 224 CPU cores available to jobs, and about 2.3 TB of RAM. You never log into it directly. Instead you ask Slurm to run something there for you (Chapter 4). Here is what Slurm's configuration says about it:

```
$ scontrol show partition hgx
PartitionName=hgx
   MaxNodes=UNLIMITED MaxTime=06:00:00 MinNodes=0 …
   Nodes=hgx-01 … State=UP TotalCPUs=256 TotalNodes=1 …
   DefMemPerCPU=1900 MaxMemPerCPU=1900
   TRES=cpu=224,mem=2321292M,node=1,billing=224
```

How to read this: one node (`hgx-01`); jobs may run for at most 6 hours (`MaxTime=06:00:00`); 224 of the 256 hardware CPU threads are handed out to jobs (`TRES=cpu=224`); 2,321,292 MB ≈ 2.3 TB of RAM; and every CPU you request comes with exactly 1900 MB of RAM (`DefMemPerCPU` / `MaxMemPerCPU`). Note that the GPUs are not listed in `TRES` . That is the quirk of Chapter 5.

3.3 The shared disk: one set of files, visible everywhere

Your home folder `/home/shovan.1` is on a network file system (**NFS**) that both machines mount. A file you create on the login node is instantly visible to a job on hgx-01, and results a job writes are instantly visible to you on the login node. That is why we could prepare scripts on the login node, run them on hgx-01, and read the logs back on the login node without copying anything. Both machines can also reach the internet, which is not true on every cluster: our setup job downloaded packages and the model directly on hgx-01.

> **WATCH OUT · /TMP IS NOT SHARED (WE TRIPPED ON THIS)**
> Only `/home` is shared; each machine has its own private `/tmp` . While writing this guide we put a helper script in the login node's `/tmp` and asked Slurm to run it on hgx-01. Job 270 failed at once with `error: execve(): /tmp/…/lab_run.sh: No such file or directory` , because hgx-01 could not see that file. Copying it into `~/MRI-MMedPO/results/lab/` fixed it (job 271). Keep everything a job needs under `/home` .

> **RECAP**
> - Login node `aihub-cluster1` : light work only. `hgx-01` : 8 B200s, reached through Slurm.
> - `/home` is shared between them; `/tmp` is not.
> - Jobs: at most 6 hours, and RAM = 1900 MB per requested CPU.

CHAPTER 4

Slurm from zero

Slurm is the program that shares hgx-01 between everyone who uses the cluster. You tell it what you need (how many CPUs, for how long), it finds room, and it runs your commands on hgx-01. Six commands cover almost everything.

4.1 The vocabulary

Job
One request to run something on the compute node, with a number Slurm assigns (our setup was job 263, the benchmark job 266). Each job has an owner, a resource request, a time limit and a state.

Partition
A named queue of machines. Ours has one partition, `hgx`, containing one node, `hgx-01`. It is the default, so `-p hgx` is optional, but we always write it to be explicit.

Node
One computer in the cluster. We have exactly one compute node.

CPUs (`-c`)
How many CPU cores the job reserves. On this cluster it also sets your RAM: 1900 MB per CPU.

Time limit (`-t`)
The longest the job may run, as `HH:MM:SS`. Slurm kills the job when it runs out. The maximum here is `06:00:00`.

Account / QOS
Who is billed and which rules apply. Yours are `panda-research` and `normal`. They are applied automatically, so you do not need to type them.

4.2 The six commands

Command	What it does
`sinfo`	Shows partitions and whether their nodes are idle, mixed (partly used), allocated (full) or down.
`squeue -u $USER`	Lists *your* queued and running jobs (`$USER` expands to your username).
`srun [options] command`	Runs a command on the compute node *now* and waits for it; you see its output live. With `--pty bash` it gives you an interactive shell on hgx-01.
`sbatch script.sbatch`	Submits a script to run *in the background*; returns immediately with a job number. Output goes to a log file.
`scancel JOBID`	Cancels a queued or running job.
`sacct -j JOBID`	Shows the history of a finished job: state, run time, exit code.

Table 3. The Slurm commands used in this project.

This is what they printed on our cluster:

```
$ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
hgx*         up    6:00:00      1   idle hgx-01
$ sbatch my_job.sbatch
Submitted batch job 272
$ squeue -u $USER
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
               272       hgx my-first shovan.1 PD       0:00      1 (None)
$ sacct -u $USER -X --starttime 2026-10-27 --format=JobID,JobName%20,Elapsed,State
JobID                    JobName    Elapsed      State
------------ -------------------- ---------- ----------
263                  mmedpo-setup   00:01:41  COMPLETED
264                     prep-data   00:00:34  COMPLETED
265                  mmedpo-smoke   00:00:30  COMPLETED
266                  mmedpo-bench   00:10:54  COMPLETED
…
```

`hgx*` : the star marks the default partition. `ST` is the state: `PD` = pending (waiting), `R` = running. `NODELIST(REASON)` shows the node when running, or the reason it is waiting when pending. `-X` in `sacct` hides the per-step detail lines.

4.3 The life of a job

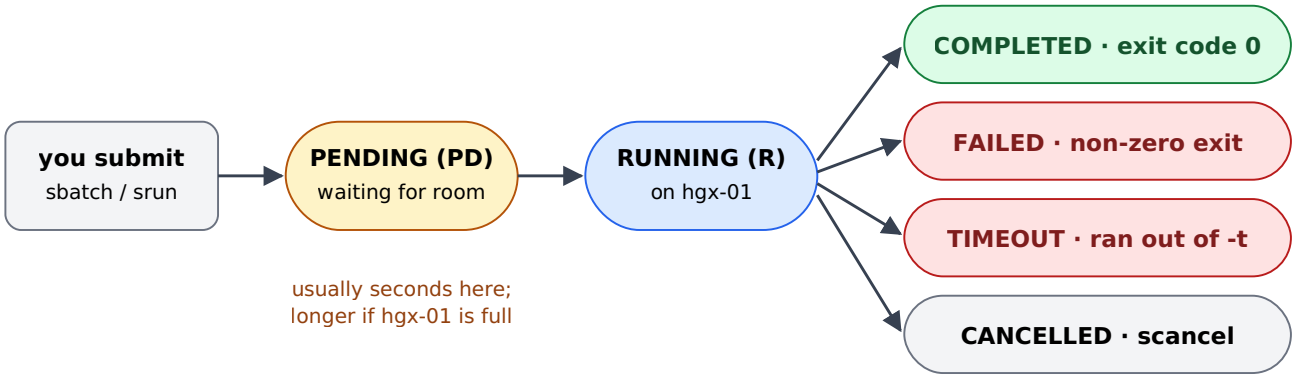


Figure 7. Every job goes PENDING → RUNNING → one final state. On our quiet cluster, jobs usually start within seconds. All our project jobs ended COMPLETED except two that failed for instructive reasons (jobs 270 and 272, see Chapter 19).

4.4 Interactive or batch?

There are two ways to run work on hgx-01, and we used both.

Interactive: `srun --pty bash`

```
[shovan.1@aihub-cluster1 ~]$ srun -p hgx -c 8
-t 01:00:00 --pty bash
[shovan.1@hgx-01 ~]$ hostname
hgx-01.aihub.lan
```

Your terminal is now *on* hgx-01 (look at the prompt). You type commands and see results immediately, like a normal shell, until you type `exit` or the time runs out. Best for exploring, quick tests and debugging. If you disconnect, the session and anything running in it end.

Batch: `sbatch script.sbatch`

```
$ sbatch scripts/run_bench.sbatch
Submitted batch job 266
$ tail -f logs/mmedpo-bench-266.out
```

You write the commands into a file and hand it over. The job runs on its own, even if you close your laptop, and writes everything it prints to a log file. Best for anything longer than a few minutes, and for anything you want to be able to repeat exactly.

4.5 Anatomy of a batch script

A batch script is an ordinary bash script with special comment lines at the top. Lines starting with `#SBATCH` are read by Slurm as options; bash itself ignores them as comments. Here is the top of our benchmark script, `scripts/run_bench.sbatch`:

scripts/run_bench.sbatch (first lines)

```
#!/bin/bash                          # run this file with bash
#SBATCH -J mmedpo-bench              # job name, shown in squeue/sacct (%x below)
#SBATCH -p hgx                       # partition
#SBATCH -c 48                        # 48 CPU cores → 48 × 1900 MB ≈ 91 GB RAM
#SBATCH -t 02:30:00                  # time limit 2 h 30 min (it needed 11 min)
#SBATCH -o /home/shovan.1/MRI-MMedPO/logs/%x-%j.out   # log file; %x = name, %j = job id
set -euo pipefail                    # stop at the first error (see below)
```

- **Ask for more time than you need, but not absurdly more.** If the limit is hit, Slurm kills the job and you lose the unfinished work. We gave the benchmark 2.5 hours for a run we expected to take about 15 minutes.
- **The log path must be absolute** and its folder must already exist. Slurm does not expand `~` or `$HOME` in `#SBATCH` lines, so we write `/home/shovan.1/…` in full. With `%x-%j` the benchmark log became `logs/mmedpo-bench-266.out`.
- **`set -euo pipefail`** makes bash stop at the first failing command (`-e`), treat unset variables as errors (`-u`), and count a pipeline as failed if any part of it fails (`pipefail`). Without this, a script carries on after an error and you get confusing results later. It has one trap, which caught us (Chapter 19).

4.6 CPUs and memory: one knob

On this cluster you do not ask for memory directly. Every CPU core comes with 1900 MB of RAM, the default and also the maximum. So the `-c` number sets both:

We asked for	Used in	RAM you get	Why that many
`-c 8`	the hands-on lab (Chapter 18)	15.2 GB	one model; RAM must hold the weights briefly while they load
`-c 16`	setup job 263	30.4 GB	parallel downloads and unzipping
`-c 48`	benchmark job 266	91.2 GB	four models loading at once (4 × ~15 GB passes through RAM) plus image decoding

Table 4. CPU requests and the RAM they bring (1900 MB per CPU). `sacct` confirms it: the lab's batch job shows `AllocCPUS 8, ReqMem 15200M`.

> **RECAP**
>
> - `sinfo` / `squeue` to look; `srun` for now-and-interactive; `sbatch` for background jobs with logs; `scancel` to stop; `sacct` for history.
> - Always give `-p hgx -c N -t HH:MM:SS`. RAM = N × 1900 MB.
> - Put `#SBATCH` lines at the top of the script, with an absolute `-o` log path.

CHAPTER 5

GPUs on this cluster: the “no GRES” quirk

On most GPU clusters you ask Slurm for a GPU, and Slurm hands you one and hides the others. Ours does not do that. Every job can see, and use, all eight GPUs. This chapter explains how we coped.

5.1 What normally happens, and what happens here

Slurm can manage “generic resources”, called **GRES**, such as GPUs. On a cluster configured that way you would write `--gres=gpu:1`, and Slurm would give your job one GPU and set an environment variable, `CUDA_VISIBLE_DEVICES`, so that your programs see only that GPU. Two people could never collide on the same card.

On hgx-01, GPUs are *not* configured as a Slurm resource. You can see this in the partition’s `TRES=cpu=224,mem=2321292M,node=1` line: there is no `gres/gpu` entry. Our very first test job confirmed it:

```
$ srun -p hgx -c 2 -t 0:02:00 --pty bash -c 'hostname; echo CVD=${CUDA_VISIBLE_DEVICES:-unset}'
hgx-01.aihub.lan
CVD=unset
```

So inside any job, all eight GPUs are visible, and a program that just says “use the GPU” takes GPU 0, whether or not someone else is already using it. If two people do that, they share one card, compete for its memory, and can crash each other with out-of-memory errors.

5.2 Our solution: look first, then choose

Before starting a model, our scripts ask `nvidia-smi` (NVIDIA’s monitoring tool) how much memory each GPU is using, and pick GPUs that are essentially empty (under 1000 MiB used):

```
$ nvidia-smi --query-gpu=index,name,memory.used,memory.total,utilization.gpu --format=csv
index, name, memory.used [MiB], memory.total [MiB], utilization.gpu [%]
0, NVIDIA B200, 0 MiB, 183359 MiB, 0 %
1, NVIDIA B200, 0 MiB, 183359 MiB, 0 %
…
7, NVIDIA B200, 0 MiB, 183359 MiB, 0 %
$ export CUDA_VISIBLE_DEVICES=$(nvidia-smi --query-gpu=index,memory.used --
format=csv,noheader,nounits | awk -F', ' '$2<1000{print $1; exit}')
$ echo "Using GPU $CUDA_VISIBLE_DEVICES"
Using GPU 0
```

Reading the long line from the inside out: `nvidia-smi … --format=csv,noheader,nounits` prints lines like `0, 0` (index, MiB used). `awk -F', '` splits each line at the comma; `'$2<1000{print $1; exit}'` prints the index of the first GPU using under 1000 MiB and stops. `$( … )` captures that output, and `export` stores it in `CUDA_VISIBLE_DEVICES` for every program started afterwards.

5.3 What `CUDA_VISIBLE_DEVICES` actually does

It is a filter and a renumbering. If you set `CUDA_VISIBLE_DEVICES=3`, a program sees exactly one GPU, and calls it `cuda:0`, even though it is physical GPU 3. This is why our timing script can always say `device="cuda:0"`: the shell decides which real card that is.

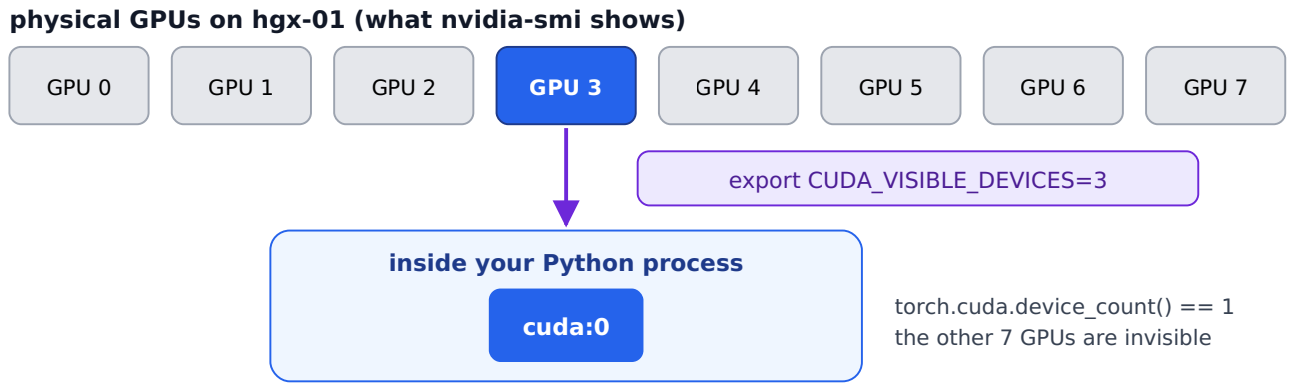


Figure 8. `CUDA_VISIBLE_DEVICES` hides every GPU not listed and renumbers the rest from 0. Setting it to 3 makes physical GPU 3 appear as `cuda:0`.

The benchmark used the same idea four times: it found four idle GPUs and started four separate Python processes with `CUDA_VISIBLE_DEVICES=0`, `=1`, `=2` and `=3`. Each process believed it had one GPU called `cuda:0` (Chapter 14).

WATCH OUT · GOOD MANNERS AND ONE SMALL RISK

- **Always check before you start**, and only take the GPUs you need. Nothing stops you from accidentally using a GPU someone else is on, except looking first.
- **Race condition:** if two people check at the same instant, both may see GPU 0 as free and both pick it. The model takes ~17 GB of a 183,359 MiB card, so two copies would still fit, just a bit slower. For big jobs, glance at `nvidia-smi` again after your model has loaded.
- If *no* GPU is idle, the variable ends up empty and your program would see no GPU at all. Our batch scripts check for this and stop with a clear message.

RECAP

- Slurm here does not hand out GPUs: every job sees all 8.
- Pick an idle one with `nvidia-smi` and set `CUDA_VISIBLE_DEVICES` yourself.
- Inside the program the chosen card is always `cuda:0`.

Part II

The software stack

The layers of software between our script and the GPU: Python and its packages, PyTorch and CUDA, and Hugging Face, where the model and data come from. This is also where the one real obstacle was: the paper's software is too old for the B200.

CHAPTER 6

Python, packages and virtual environments

The model code is written in Python. Python on its own is small: the real work is done by add-on *packages*, and they must be exactly the right versions. A *virtual environment* is how we keep those versions together in one place.

6.1 Python and packages

Python is both a programming language and the program (`python`) that runs code written in it. A **package** (or library) is a bundle of ready-made code someone else wrote, for example PyTorch for GPU maths or transformers for model building blocks. Packages are downloaded from the **Python Package Index (PyPI)**, or for PyTorch from PyTorch's own server, and installed with a package manager. Packages depend on other packages, so installing a dozen of them pulled in many more: our environment ended up with 56 installed packages.

The cluster's built-in Python is version 3.9 and has no package manager (`pip`) and no `conda` . Rather than ask an administrator to install things, we used **uv**, a fast, self-contained package manager that lives in your home folder (`~/.local/bin/uv`). uv can even download a private copy of Python itself.

6.2 Why exact versions matter

Research code is written against the versions its authors happened to have. Libraries change: functions get renamed, defaults change, internal details move. The LLaVA code inside MMedPO reaches into the internals of the transformers library, so it only works with a narrow range of transformers versions. That is why everything below is pinned to an exact version (`==`) or an upper bound (`<`).

Package	Paper's `requirements.txt`	We used	Why
Python	3.10	3.10.22	Same as the authors.
torch	**2.1.2**	**2.8.0 (+cu128)**	**2.1.2 cannot run on a B200 (Chapter 7, tested in job 275).**
torchvision	(not pinned)	0.23.0	The release that matches torch 2.8.0.
transformers	4.33.0	4.37.2	Newer torch needed a newer transformers. 4.37.2 is the newest release the LLaVA-1.5 model code still runs on unmodified.
tokenizers	0.13.3	0.15.2	The version that transformers 4.37.2 expects.
peft (LoRA)	0.14.0	0.11.1	A peft release that works with transformers 4.37.
accelerate	0.21.0	0.28.0	Used for memory-efficient loading; matched to transformers 4.37.

Package	Paper's `requirements.txt`	We used	Why
numpy	1.22.4	< 2	numpy 2 broke compatibility with older compiled packages.
huggingface_hub	(not pinned)	< 0.26, with `hf_transfer`	Downloads; an API that transformers 4.37 understands.

Table 5. The package versions that matter. We only needed inference, so we installed only what inference needs, not the paper's full training list (deepspeed, trl, wandb and so on).

The harmless `FutureWarning: `resume_download` is deprecated` line at the top of every log comes from this deliberate mix of older and newer packages. It is a warning, not an error.

6.3 Virtual environments: a private toolbox per project

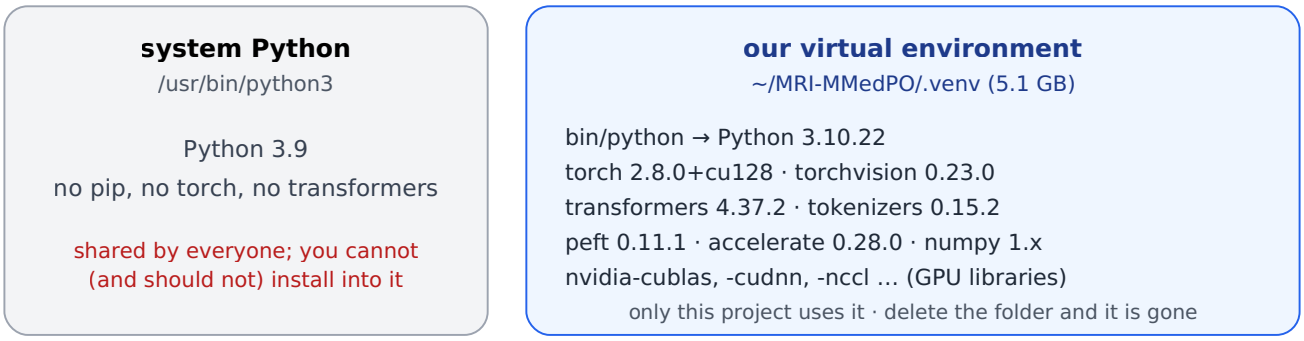


Figure 9. A virtual environment is a folder containing its own Python and its own packages. Different projects can have different, even conflicting, versions without interfering with each other or with the system.

A **virtual environment** ("venv") is just a folder, ours is `~/MRI-MMedPO/.venv`, with its own `python` and its own installed packages. There are two ways to use it:

1. **Activate it** with `source .venv/bin/activate`. From then on, in that terminal, typing `python` runs the venv's Python. This is handy interactively:

```
$ cd ~/MRI-MMedPO
$ source .venv/bin/activate
(.venv) $ which python
/home/shovan.1/MRI-MMedPO/.venv/bin/python
```

2. **Call its Python by full path**, `/home/shovan.1/MRI-MMedPO/.venv/bin/python script.py`. Nothing needs to be activated, which makes it the safer choice inside batch scripts. That is what `run_bench.sbatch` does.

6.4 Building it with uv

Three commands from the setup script built the whole environment. The first downloads Python 3.10 and creates the folder; the next two install packages:

```
$UV venv --python 3.10 .venv
$UV pip install --index-url https://download.pytorch.org/whl/cu128 torch==2.8.0
torchvision==0.23.0
$UV pip install "transformers==4.37.2" "tokenizers==0.15.2" "peft==0.11.1"
"accelerate==0.28.0" \
    "numpy<2" sentencepiece protobuf shortuuid einops einops-exts pillow pandas pyarrow tqdm \
    "huggingface_hub[hf_transfer]<0.26"
```

The log of setup job 263 shows what happened, including why the environment is 5.1 GB. PyTorch brings NVIDIA's GPU libraries with it as separate packages, several of them over half a gigabyte:

```
Downloading cpython-3.10.22-linux-x86_64-gnu (download) (28.0MiB)
Using CPython 3.10.22
Creating virtual environment at: .venv
Resolved 28 packages in 632ms
Downloading torch (848.0MiB)
Downloading nvidia-cudnn-cu12 (674.0MiB)
Downloading nvidia-cublas-cu12 (566.8MiB)
Downloading nvidia-nccl-cu12 (307.4MiB)
…
 + tokenizers==0.15.2
 + transformers==4.37.2
torch 2.8.0+cu128 cuda 12.8 | transformers 4.37.2 | peft 0.11.1
```

The last line is a one-line self-test at the end of the install step: it imports the three key packages and prints their versions, so a broken install shows up immediately rather than later in the benchmark.

> **RECAP**
>
> - Packages must be specific versions. The LLaVA code needs transformers ≈ 4.37, and the B200 needs torch ≥ 2.7.
> - Our venv is `~/MRI-MMedPO/.venv`. Activate it with `source .venv/bin/activate`, or call `.venv/bin/python` directly.
> - uv built it in one job, downloading Python 3.10.22 and about 5 GB of packages.

CHAPTER 7

PyTorch, CUDA and why the B200 needed newer versions

This chapter explains the one real obstacle we hit. The software the paper used cannot run on our GPUs at all. Understanding why tells you a lot about how the GPU software stack fits together.

7.1 The layers between our script and the chip

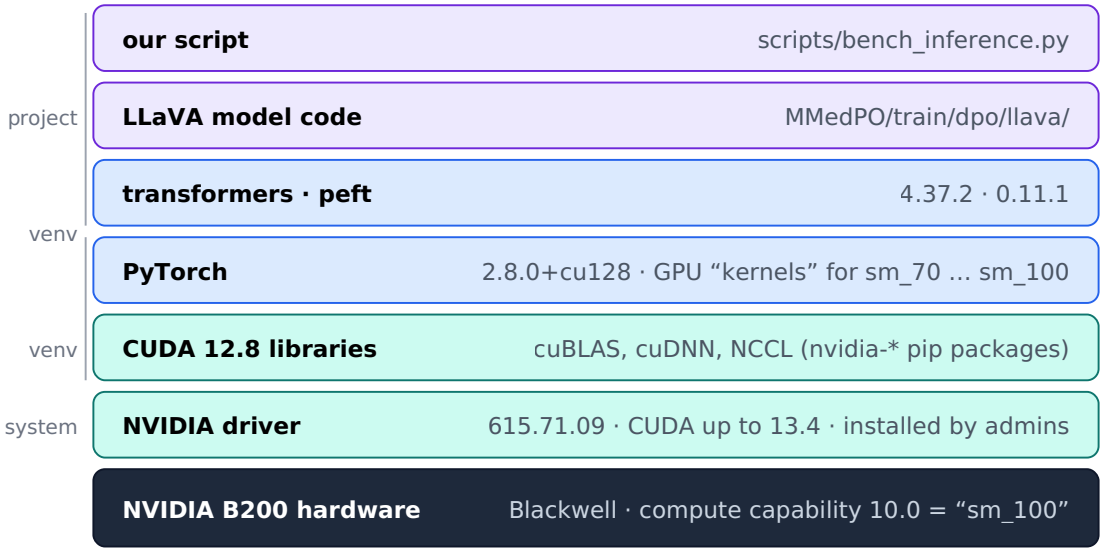


Figure 10. Each layer talks only to the one below it. Every layer from transformers down to the CUDA libraries lives inside our virtual environment; only the driver is installed system-wide. The PyTorch layer is where the paper’s version breaks.

CUDA
: NVIDIA’s platform for running general-purpose programs on its GPUs: a programming model, a compiler, and a set of maths libraries (cuBLAS for matrix multiplication, cuDNN for neural-network operations, NCCL for multi-GPU communication).

Kernel
: A small program that runs on the GPU, for example “multiply these two matrices”. PyTorch contains thousands of them, compiled ahead of time.

Compute capability (sm_XX)
: NVIDIA’s version number for a GPU’s instruction set. Kernels are compiled for specific capabilities. A100 = 8.0 (`sm_80`), H100 = 9.0 (`sm_90`), B200 = 10.0 (`sm_100`).

Driver
: The system-level software that lets programs talk to the GPU. It must be at least as new as the CUDA version a program

was built for. Ours supports CUDA 13.4, newer than the 12.8 our PyTorch uses, so that is fine.

7.2 What happens with the paper's PyTorch 2.1.2

PyTorch 2.1.2 was released in December 2023, before Blackwell GPUs existed, so it contains no kernels for `sm_100`. We did not want to just assert that, so while writing this guide we installed it in a throw-away environment and tried a one-line GPU calculation on hgx-01 (job 275):

```
$ python -c "import torch; …; x = torch.ones(3, device='cuda'); print(x * 2)"
UserWarning:
NVIDIA B200 with CUDA capability sm_100 is not compatible with the current PyTorch installation.
The current PyTorch install supports CUDA capabilities sm_50 sm_60 sm_70 sm_75 sm_80 sm_86 sm_90.
…
RuntimeError: CUDA error: no kernel image is available for execution on the device
```

Notice what *did* work. PyTorch 2.1.2 found the GPU (`torch.cuda.is_available()` was `True`, and it printed the name "NVIDIA B200"). It failed the moment it tried to run actual arithmetic, because it had no kernel the B200 can execute. This is the most important compatibility fact in the whole project: **the paper's exact environment cannot run on our hardware**, no matter how much memory there is.

Our PyTorch, by contrast, lists `sm_100` among its compiled targets:

```
$ python -c "import torch; print(torch.__version__, torch.version.cuda,
torch.cuda.get_device_capability(0), torch.cuda.get_arch_list())"
2.8.0+cu128 12.8 (10, 0) ['sm_70', 'sm_75', 'sm_80', 'sm_86', 'sm_90', 'sm_100', 'sm_120']
```

PyTorch is released in several builds for different CUDA versions. The "+cu128" in "2.8.0+cu128" means "built against CUDA 12.8". That is why the install command points at a special address, `--index-url https://download.pytorch.org/whl/cu128`: that server hosts the CUDA-12.8 build, which is the one that includes Blackwell kernels.

> **WATCH OUT · DOES UPGRADING PYTORCH CHANGE THE RESULTS?**
> In principle, slightly. Different library versions can round floating-point numbers in a different order, and the model samples with a small amount of randomness (temperature 0.2), so individual answers may differ from the authors' runs. The model's weights are identical, though, and our closed-question accuracies came out close to the paper's: 73.6% vs 73.08% on SLAKE, 64.9% vs 66.54% on VQA-RAD (Chapter 16). For *timing*, which is what we were asked, the version change does not matter: the old version simply cannot run here.

7.3 A one-time warm-up cost

One more GPU-software effect shows up in our numbers. In every run on 28 October the *first* question took about 15 seconds, and every later one took a fraction of a second:

```
[1/451] 15.151s 4 tok | 'No'
[2/451] 0.063s 4 tok | 'Yes'
[3/451] 0.068s 4 tok | 'No'
```

The first time a program uses a particular GPU operation, some GPU code is prepared or compiled for the exact chip and its libraries are initialised. NVIDIA caches compiled code on disk, in `~/.nv/ComputeCache`. On our system that folder is 223 MB, and its files were written during our first GPU jobs on 28 October (22:16–22:18), which matches. In the lab run a day later, with the cache already filled, the first question took 1.45 s instead of 15 s. Either way, this is a one-time cost per program run, so **our script times the first 3 samples but leaves them out of the averages** (the "warm-up", Chapter 13).

> **RECAP**
>
> - GPU code is compiled for specific chip generations. B200 = `sm_100`.
> - The paper's torch 2.1.2 has no `sm_100` kernels. It fails with "no kernel image is available" (job 275).
> - torch 2.8.0+cu128 supports it. That required newer transformers, tokenizers, peft and accelerate.
> - The first GPU call of a run is slow (≈ 15 s cold, ≈ 1.5 s with the cache), so it is excluded from timing.

CHAPTER 8

Hugging Face: where the model and data come from

Nobody emails 15 GB of model weights. They are published on the Hugging Face Hub, a public library of models and datasets, and downloaded with a couple of lines of Python.

8.1 The Hub

The **Hugging Face Hub** (huggingface.co) works like GitHub, but for models and datasets. Each **repository** has a name of the form `owner/name` and holds files: weights, configuration, a README. Downloading a repository gives you an exact copy of those files. We used six repositories:

Repository	What it is	On our disk
`microsoft/llava-med-v1.5-mistral-7b`	The base model: LLaVA-Med v1.5 (Mistral-7B + CLIP + projector)	15 GB
`openai/clip-vit-large-patch14-336`	The CLIP vision encoder's configuration and weights, which LLaVA's code loads by name	1.6 GB (hf_cache)
`zky11235/mmedpo_checkpoints`	**The authors' released MMedPO adapters, one per dataset: iuxray, mimic, slake, vqa-rad**	**2.6 GB**
`flaviagiammarino/vqa-rad` (dataset)	VQA-RAD test split, one Parquet file with images inside	149 MB
`BoKelvin/SLAKE` (dataset)	SLAKE questions (`test.json`) and images (`imgs.zip`)	477 MB
`dz-osamu/IU-Xray` (dataset)	IU-Xray reports (`test.jsonl`) and X-ray images (`image.zip`)	2.2 GB

Table 6. What we downloaded. Sizes are from `du -sh` on our disk; the dataset sizes include the unzipped images.

8.2 Two ways files are stored locally

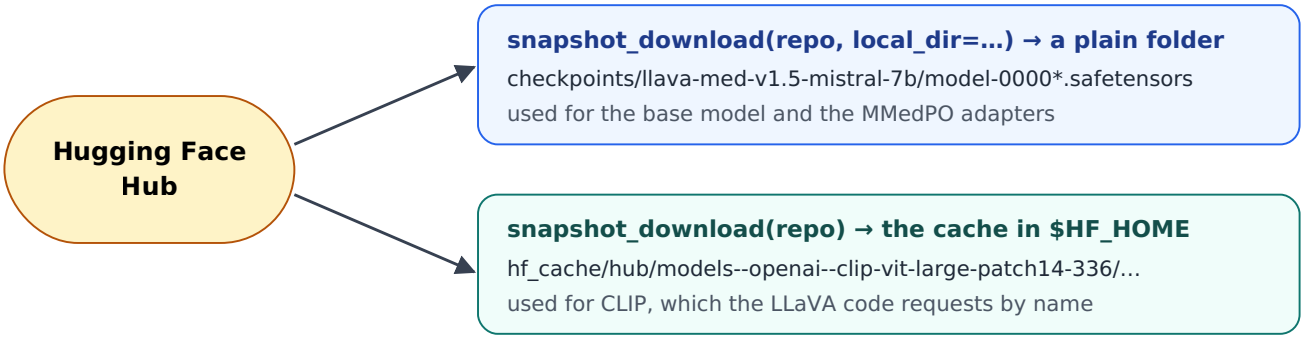


Figure 11. Downloading with an explicit `local_dir` gives an ordinary folder. Downloading without one puts files in the Hugging Face cache, where code that asks for a model by name will find them automatically.

The key code from the setup script is short:

scripts/setup_env.sbatch (weights)

```
from huggingface_hub import snapshot_download
W = "/home/shovan.1/MRI-MMedPO"
snapshot_download("microsoft/llava-med-v1.5-mistral-7b", local_dir=f"{W}/checkpoints/llava-med-v1.5-mistral-7b")
snapshot_download("openai/clip-vit-large-patch14-336", allow_patterns=["*.json", "*.txt", "pytorch_model.bin"])
snapshot_download("zky11235/mmedpo_checkpoints", local_dir=f"{W}/checkpoints/mmedpo_checkpoints")
```

`allow_patterns` downloads only the files we need from the CLIP repository (the PyTorch weights and configs), skipping duplicate copies of the same weights in other formats.

8.3 Three environment variables that control downloads

Variable	Set where	Effect
`HF_HOME=$W/hf_cache`	both scripts	Puts the Hugging Face cache in the project folder instead of `~/.cache`, so everything for this project lives in one place.
`HF_HUB_ENABLE_HF_TRANSFER=1`	setup	Uses `hf_transfer`, a downloader that opens many parallel connections. All weights and data (about 20 GB) came down within the 1 min 41 s setup job.
`HF_HUB_OFFLINE=1`	benchmark	Forbids network access. The model must already be on disk, and no time is spent checking for updates, so network delays cannot leak into the timings.

Table 7. Hugging Face settings we used.

WHY MIMIC-CXR IS MISSING

Two of the paper's four test sets are freely downloadable. MIMIC-CXR is not. It contains real hospital records, and PhysioNet only gives access to people who complete a human-subjects-research training course and sign a data use agreement. We did not have those credentials. So we timed the authors' *MIMIC checkpoint* on IU-Xray chest X-rays instead (Chapter 14). It is the same model and the same kind of task, on a different set of X-rays. You decided on 28 October that this stand-in was enough for now.

RECAP

- Models and datasets come from the Hugging Face Hub as repositories of files.
- `snapshot_download(local_dir=…)` gives a plain folder; without it, files go to the cache in `$HF_HOME`.
- Download once in the setup job; run with `HF_HUB_OFFLINE=1` so timings never include network time.

Part III

Our pipeline, step by step

Every file we wrote and every job we ran, in order, with the real code and the real output. By the end of this part you will be able to read each script line by line.

CHAPTER 9

The map: folders, files and the six steps

Everything for this project lives in one folder, `~/MRI-MMedPO`, and the whole run from an empty folder to final numbers took four Slurm jobs and about fifteen minutes. Here is the map before we walk through each step.

9.1 The project folder

~/MRI-MMedPO/	
├── MMedPO/	the authors' code (git clone), 12 MB
├── .venv/	**5.1 GB** Python 3.10 + packages (Ch. 6)
├── checkpoints/	
│ ├── llava-med-v1.5-mistral-7b/	**15 GB** base model LLaVA-Med v1.5 (Ch. 12)
│ ├── mmedpo_checkpoints/	**2.6 GB** 4 MMedPO adapters (Ch. 12)
│ └── llava-med-1.5_*_mmedpo_lora	4 shortcuts with "lora" in the name (Ch. 12)
├── hf_cache/	**1.6 GB** CLIP vision encoder (Ch. 8)
├── data/	
│ └── vqa-rad/ slake/ iuxray/	149 MB · 477 MB · **2.2 GB** test sets (Ch. 11)
├── scripts/	our 5 scripts, 50 KB (Ch. 10–15)
├── logs/	Slurm and per-run logs
├── results/	
│ └── smoke/ full/ lab/	answers + timing summaries (Parts IV, V)
├── report/	the HTML report sent to the advisor
├── guides/	these two guides and their sources
└── .uv_cache/	**5.2 GB** uv download cache (safe to delete)

Figure 12. The project folder with real sizes (`du -sh`). Almost all the space is the model (17 GB in `checkpoints/`) and the Python environment.

Our own code is just five files in `scripts/` :

File	Kind	Job
`setup_env.sbatch`	Slurm batch script	Build the Python environment; download the model, adapters and test sets.
`prepare_testsets.py`	Python	Convert the three test sets into one simple question-file format.
`bench_inference.py`	Python	Load one model, answer every question in a file, and time every step.
`run_bench.sbatch`	Slurm batch script	Run `bench_inference.py` four times in parallel on four GPUs.
`summarize.py`	Python	Collect the four summaries into one table.

Table 8. Our scripts. Everything else in the folder was downloaded or generated.

9.2 The six steps and the jobs that ran them

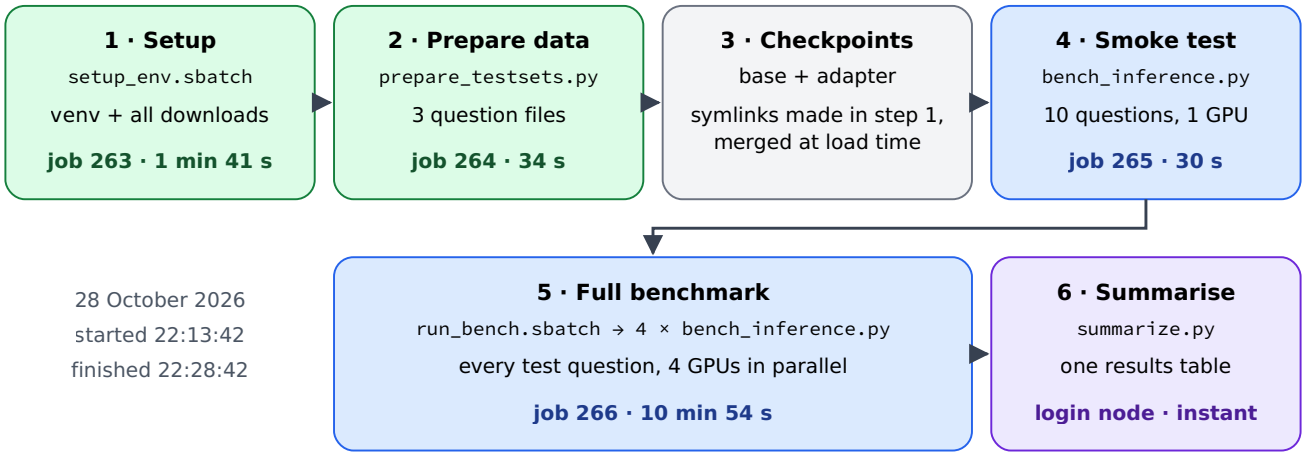


Figure 13. The pipeline. Green steps prepare things, blue steps run the model, purple turns results into a table. A smoke test, a 10-question trial run, came before the full run so that a mistake would cost 30 seconds rather than 10 minutes.

> **KEY IDEA · ALWAYS SMOKE-TEST FIRST**
>
> Before committing to a long run, run the same code on a tiny slice of the data (`--limit 10`). It catches wrong paths, missing files and crashes in seconds. Our smoke test (job 265) also gave the first real numbers: a 17.3 s model load, 16.3 GB of weights in GPU memory, and about 0.17 s per question.

CHAPTER 10

Step 1 · Build the environment and download everything

`scripts/setup_env.sbatch` turns an empty folder into one that has everything needed to run the model. It uses no GPU, but it runs as a Slurm job anyway, because a 5 GB install and about 20 GB of downloads are not for the login node.

10.1 The header and settings

scripts/setup_env.sbatch (lines 1–12)

```
#!/bin/bash
#SBATCH -J mmedpo-setup
#SBATCH -p hgx
#SBATCH -c 16
#SBATCH -t 01:30:00
#SBATCH -o /home/shovan.1/MRI-MMedPO/logs/%x-%j.out
# Builds the Python env and downloads model weights + test sets (no GPU work).
set -euo pipefail
W=/home/shovan.1/MRI-MMedPO
UV=/home/shovan.1/.local/bin/uv
export HF_HOME=$W/hf_cache UV_CACHE_DIR=$W/.uv_cache HF_HUB_ENABLE_HF_TRANSFER=1
cd $W
```

- `-c 16` gives 16 CPUs and 30.4 GB of RAM for parallel downloads and unzipping. `-t 01:30:00` was a generous guess; it needed 1 min 41 s.
- `W` and `UV` are shell variables, used to avoid typing long paths. `$W` later expands to `/home/shovan.1/MRI-MMedPO`.
- `export` makes the variables visible to every program the script starts: Python, uv and the Hugging Face library all read them. `UV_CACHE_DIR` keeps uv's download cache in the project folder too.

10.2 The Python environment

scripts/setup_env.sbatch (lines 14–23)

```
# Paper used torch 2.1.2, which has no Blackwell (B200, sm_100) kernels; torch 2.8 + cu128 does.
# transformers 4.37.2 is the last release the LLaVA-1.5 model code runs on unmodified.
$UV venv --python 3.10 .venv
source .venv/bin/activate
$UV pip install --index-url https://download.pytorch.org/whl/cu128 torch==2.8.0 torchvision==0.23.0
$UV pip install "transformers==4.37.2" "tokenizers==0.15.2" "peft==0.11.1" "accelerate==0.28.0" \
    "numpy<2" sentencepiece protobuf shortuuid einops einops-exts pillow pandas pyarrow tqdm \
    "huggingface_hub[hf_transfer]<0.26"
python -c "import torch, transformers, peft; print('torch', torch.__version__, …)"
```

Chapter 6 explained these lines. Two details: PyTorch is installed in its own command because it comes from a different server (`--index-url`); and the backslash `\` at the end of a line means “the command continues on the next line”.

10.3 Weights and the “lora” shortcuts

scripts/setup_env.sbatch (lines 25–37)

```
python - <<'EOF'
from huggingface_hub import snapshot_download
W = "/home/shovan.1/MRI-MMedPO"
snapshot_download("microsoft/llava-med-v1.5-mistral-7b", local_dir=f"{W}/checkpoints/llava-med-v1.5-
mistral-7b")
snapshot_download("openai/clip-vit-large-patch14-336", allow_patterns=[…])
snapshot_download("zky11235/mmedpo_checkpoints", local_dir=f"{W}/checkpoints/mmedpo_checkpoints")
EOF
# LLaVA's loader only takes the LoRA-merge path when the folder name contains "lora".
for ds in iuxray mimic slake vqa-rad; do
  ln -sfn $W/checkpoints/mmedpo_checkpoints/llava-med-1.5_${ds}_mmedpo $W/checkpoints/llava-med-1.5_${ds}
_mmedpo_lora
done
```

- **`python - <<'EOF' … EOF`** is a “here-document”: every line up to the line `EOF` is fed to Python as a script. It lets a few lines of Python live inside a bash file. The quotes around `'EOF'` tell bash not to touch `$` signs inside. (Here-documents have one trap, which caught us while writing this guide: if the text itself contains a line that is exactly `EOF`, the here-document ends there. Pick an end-marker that cannot appear in the text.)
- **The `for` loop** runs its body four times, with `$ds` set to each dataset name in turn. `ln -sfn TARGET LINK` creates a **symbolic link**: a shortcut named `LINK` that points to the folder `TARGET`. Chapter 12 explains why we need shortcuts whose names contain “lora”.

10.4 Test sets

scripts/setup_env.sbatch (lines 39–52)

```
python - <<'EOF'
from huggingface_hub import hf_hub_download
D = "/home/shovan.1/MRI-MMedPO/data"
hf_hub_download("flaviagiammarino/vqa-rad", "data/test-00000-of-00001-e5bc3d208bb4deeb.parquet", repo_type="da
taset", local_dir=f"{D}/vqa-rad")
for f in ["test.json", "imgs.zip"]:
    hf_hub_download("BoKelvin/SLAKE", f, repo_type="dataset", local_dir=f"{D}/slake")
for f in ["test.jsonl", "image.zip"]:
    hf_hub_download("dz-osamu/IU-Xray", f, repo_type="dataset", local_dir=f"{D}/iuxray")
EOF
cd $W/data/slake && unzip -qo imgs.zip
cd $W/data/iuxray && unzip -qo image.zip
du -sh $W/checkpoints $W/hf_cache $W/data
echo SETUP_DONE
```

`hf_hub_download` fetches single files, where `snapshot_download` fetches whole repositories. `unzip -qo` extracts quietly (`-q`) and overwrites without asking (`-o`), which matters because a batch job cannot answer questions. `A && B` means "run B only if A succeeded". The final `echo SETUP_DONE` is a marker: if it is the last line of the log, every step before it worked.

10.5 Running it and watching it

```
$ sbatch scripts/setup_env.sbatch
Submitted batch job 263
$ tail -f logs/mmedpo-setup-263.out       # follow the log live; Ctrl-C stops watching, not the job
…
torch 2.8.0+cu128 cuda 12.8 | transformers 4.37.2 | peft 0.11.1
17G     /home/shovan.1/MRI-MMedPO/checkpoints
1.6G    /home/shovan.1/MRI-MMedPO/hf_cache
2.6G    /home/shovan.1/MRI-MMedPO/data
SETUP_DONE
```

Job 263 ran from 22:13:42 to 22:15:23 on 28 October. It is safe to run again: uv reuses what is already installed, the Hugging Face functions skip files that are already complete, and `ln -sfn` and `unzip -o` overwrite. A script you can safely re-run is called **idempotent**, and that is a good habit for setup scripts.

CHAPTER 11

Step 2 · Turn the test sets into question files

The three test sets arrived in three different formats. `scripts/prepare_testsets.py` converts them all into one simple format that the timing script can read: one question per line.

11.1 The target format: JSON Lines

JSON is a common text format for structured data: `{"key": value, …}`. A **JSON Lines** file (`.jsonl`) has one JSON object per line, here one question. These are the first lines of each of our three files:

data/vqa-rad/test.jsonl (451 lines)

```
{"question_id": 0, "image": "0000.png", "question": "is there evidence of an aortic
aneurysm?", "answer": "yes", "answer_type": "CLOSED"}
```

data/slake/test.jsonl (1061 lines)

```
{"question_id": 11934, "image": "xmlab102/source.jpg", "question": "What modality is used to
take this image?", "answer": "CT", "answer_type": "OPEN"}
```

data/iuxray/test_mmedpo.jsonl (590 lines)

```
{"question_id": 0, "image": "CXR3030_IM-1405/0.png", "gt_report": "Normal cardiomediastinal
silhouette. There is no focal consolidation. There are no XXXX of a large pleural effusion.
There is no pneumothorax. There is no acute bony abnormality seen."}
```

Each line says which image to open (relative to an image folder), what to ask, and the reference answer (`answer`, or `gt_report` for "ground-truth report"). The timing script never looks at the reference answer except for a rough accuracy check at the end.

11.2 What the script does for each dataset

scripts/prepare_testsets.py (VQA-RAD part)

```
df = pd.read_parquet(f"{D}/vqa-rad/data/test-00000-of-00001-e5bc3d208bb4deeb.parquet")
for i, r in df.iterrows():
    name = f"{i:04d}.png"
    Image.open(io.BytesIO(r["image"]["bytes"])).save(f"{D}/vqa-rad/images/{name}")
    ans = str(r["answer"]).strip()
    rows.append({…, "answer_type": "CLOSED" if ans.lower() in ("yes", "no") else "OPEN"})
```

- **VQA-RAD** came as a **Parquet** file, a compact table format, with each image stored *inside* the table as raw bytes. The script unpacks every image to a PNG file (`0000.png`, `0001.png`, …) and

marks a question CLOSED if its answer is yes or no, which is how LLaVA-Med's evaluation splits closed from open questions.

- **SLAKE** is bilingual (English and Chinese). The script keeps only English questions (`q_lang == "en"`), giving 1061, and keeps SLAKE's own OPEN/CLOSED labels.
- **IU-Xray** studies usually have two X-ray views (front and side). MMedPO's report script feeds the model one image, so we keep the first view (`0.png`). There is no question: the prompt is chosen by the timing script. The 590-study test split is the one MMedPO uses (from the R2GenGPT project).

> **WHAT IS "XXXX"?**
> IU-Xray reports are real radiology reports from Indiana University hospitals. Before publication, anything that might identify a patient (names, dates and some other words) was automatically replaced with `XXXX` . This is called **de-identification**. The replacement was over-eager, so ordinary words were sometimes masked too ("There are no XXXX of a large pleural effusion"). It is part of the dataset, not an error in our processing.

This script ran as job 264 (4 CPUs, 34 s). It reads and writes thousands of image files, so it belongs in a job, not on the login node. It prints one line per file it writes, for example `…/data/vqa-rad/test.jsonl: 451 items` .

CHAPTER 12

Step 3 · Checkpoints: base model plus MMedPO adapter

The MMedPO authors did not publish a full copy of the model for each dataset. They published small *adapters* that must be added to the base model. This chapter shows what those files are, how the loader combines them, and the one naming quirk we had to work around.

12.1 What the authors released

Guide 1 explained LoRA: instead of changing a 4096 × 4096 weight matrix W directly, training learns two thin matrices B (4096 × r) and A (r × 4096) whose product is the change. Only A and B are saved, and that saved set is the **adapter**. Here is one of the four:

```
$ ls -la checkpoints/mmedpo_checkpoints/llava-med-1.5_iuxray_mmedpo/
-rw-r--r-- 1 shovan.1 shovan.1       792 Oct 28 22:14 adapter_config.json
-rw-r--r-- 1 shovan.1 shovan.1 708925520 Oct 28 22:14 adapter_model.safetensors
-rw-r--r-- 1 shovan.1 shovan.1      1566 Oct 28 22:14 config.json
-rw-r--r-- 1 shovan.1 shovan.1       912 Oct 28 22:14 non_lora_trainables.bin
-rw-r--r-- 1 shovan.1 shovan.1    359446 Oct 28 22:14 trainer_state.json
…
$ cat checkpoints/mmedpo_checkpoints/llava-med-1.5_iuxray_mmedpo/adapter_config.json
{ …
  "lora_alpha": 256,
  "lora_dropout": 0.05,
  "peft_type": "LORA",
  "r": 128,
  "target_modules": ["gate_proj", "v_proj", "up_proj", "o_proj", "q_proj", "down_proj", "k_proj"],
  … }
```

- `adapter_model.safetensors` (709 MB) holds the A and B matrices. That is about 5% of the base model's 15.1 GB.
- `adapter_config.json`: rank r = 128 and scale α = 256, exactly the paper's settings. LoRA was applied to all seven kinds of matrix in every transformer layer: the attention projections (`q_proj`, `k_proj`, `v_proj`, `o_proj`) and the feed-forward ones (`gate_proj`, `up_proj`, `down_proj`).
- `non_lora_trainables.bin` is meant for any non-LoRA parts that were trained, such as the image projector. At 912 bytes it is essentially empty.
- `trainer_state.json` is a training log (loss per step). It is not needed for inference, but is interesting to look at.

12.2 Merging: adapter in, overhead out

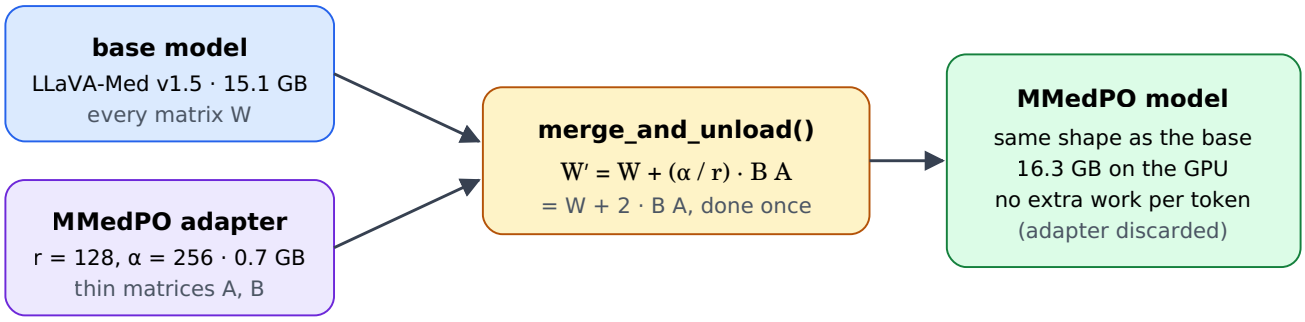


Figure 14. At load time the adapter is folded into the base weights once. The result is an ordinary model of exactly the original size and shape, so MMedPO adds no inference cost compared with plain LLaVA-Med. (That follows from the shapes; we did not time plain LLaVA-Med separately.)

The LLaVA loader (`MMedPO/train/dpo/llava/model/builder.py`) does this in a few steps, and you can see each one announced in our logs:

builder.py (condensed)

```
if 'lora' in model_name.lower() and model_base:
  print('Loading LLaVA from base model...')
  model = LlavaLlamaForCausalLM
          .from_pretrained(model_base, …)
  print('Loading additional LLaVA weights...')
  … load non_lora_trainables.bin …
  print('Loading LoRA weights...')
  model = PeftModel.from_pretrained(
          model, model_path)
  print('Merging LoRA weights...')
  model = model.merge_and_unload()
print('Model is loaded...')
```

logs/bench_vqa-rad.log

```
Loading LLaVA from base model...
Loading checkpoint shards: 100%|██████|
  4/4 [00:11<00:00, 2.84s/it]
Loading additional LLaVA weights...
Loading LoRA weights...
Merging LoRA weights...
Model is loaded...
```

“Loading checkpoint shards 4/4” is the four safetensors files of the base model being read. Merging happens after loading, and only once.

12.3 The naming quirk, and why we made shortcuts

Look at the first line of the loader: it only takes the “base + adapter, then merge” path **if the word “lora” appears in the model’s folder name**. The name comes from `get_model_name_from_path`, which takes the last part of the path. The authors’ folders are called `llava-med-1.5_iuxray_mmedpo` and so on, with no “lora” in them. Pointed at those folders directly, the loader would take a different code path that expects a complete model rather than an adapter, and would not apply MMedPO correctly.

Renaming the downloaded folders would work, but it is fragile, because a re-download would bring back the old names. Instead, the setup script created **symbolic links** (shortcuts) whose names end in `_lora`:

```
$ ls -l checkpoints
llava-med-1.5_iuxray_mmedpo_lora -> …/checkpoints/mmedpo_checkpoints/llava-med-1.5_iuxray_mmedpo
llava-med-1.5_mimic_mmedpo_lora -> …/checkpoints/mmedpo_checkpoints/llava-med-1.5_mimic_mmedpo
llava-med-1.5_slake_mmedpo_lora -> …/checkpoints/mmedpo_checkpoints/llava-med-1.5_slake_mmedpo
llava-med-1.5_vqa-rad_mmedpo_lora -> …/checkpoints/mmedpo_checkpoints/llava-med-1.5_vqa-rad_mmedpo
llava-med-v1.5-mistral-7b
mmedpo_checkpoints
```

We always pass the `_lora` shortcut as `--model-path` and the base folder as `--model-base`. The loader sees “lora” in the name, loads the base, and merges the adapter, which the “Loading LoRA weights… Merging LoRA weights…” lines in every log confirm.

WATCH OUT · HOW LONG LOADING TAKES DEPENDS ON WHAT ELSE IS HAPPENING

We saw three very different load times for the same model: 17.3 s when one model loaded alone (smoke test, 28 Oct); 36–45 s when four loaded at the same moment (benchmark), because four processes were reading 4 × 15 GB from the network disk at once; and 5.4–5.5 s in the lab on 29 Oct, when the files were still cached in the compute node’s memory from earlier runs. None of this affects the per-question times, which are measured after loading.

RECAP

- MMedPO = base LLaVA-Med (15.1 GB) + a 709 MB LoRA adapter per dataset (r = 128, α = 256).
- The loader merges the adapter into the weights once, so there is no inference overhead.
- The loader only merges if the folder name contains “lora”, hence our `_lora` symbolic links.

CHAPTER 13

Step 4 · The timing script, line by line

`scripts/bench_inference.py` is the heart of the measurement. It loads one model, then for every question in a file builds the input exactly as the authors' own inference scripts do, generates an answer, and records a timestamp at every stage. Everything it adds on top of the authors' code is bookkeeping.

13.1 How you call it

```
$ python scripts/bench_inference.py --task vqa --tag lab-vqa-rad \
    --model-path checkpoints/llava-med-1.5_vqa-rad_mmedpo_lora \
    --model-base checkpoints/llava-med-v1.5-mistral-7b \
    --question-file data/vqa-rad/test.jsonl --image-folder data/vqa-rad/images \
    --out-dir results/lab --limit 10
```

Option	Meaning
`--task vqa | report`	Answer a question (cap 64 new tokens) or write a report (cap 1024 tokens, prompt chosen by the script).
`--model-path`, `--model-base`	The adapter (via its `_lora` shortcut) and the base model (Chapter 12).
`--question-file`, `--image-folder`	A `.jsonl` file from Chapter 11, and the folder its `image` names are relative to.
`--out-dir`, `--tag`	Where to write *tag*`_answers.jsonl` and *tag*`_summary.json`.
`--limit N`	Only the first N questions (0 = all). Ideal for smoke tests.
`--warmup 3`	How many initial samples to leave out of the averages (default 3).
`--temperature 0.2`	Sampling temperature, the same as the authors' scripts (explained below).

Table 9. Command-line options of `bench_inference.py`.

13.2 Part 1: load the model and measure memory

scripts/bench_inference.py (loading)

```
sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "MMedPO", "train", "dpo"))
from llava.model.builder import load_pretrained_model     # the authors' loader
…
set_seed(0); random.seed(42)                              # reproducible sampling and prompt choice
t0 = time.perf_counter()
tokenizer, model, image_processor, _ = load_pretrained_model(
    args.model_path, args.model_base, model_name, device="cuda:0")
torch.cuda.synchronize()
load_s = time.perf_counter() - t0                         # → "model_load_s"
weights_gb = torch.cuda.memory_allocated() / 1e9          # → "weights_gpu_mem_gb" (16.28)
torch.cuda.reset_peak_memory_stats()
```

The first line tells Python where to find the authors' `llava` package, so we use their model code unchanged. `time.perf_counter()` is a high-resolution stopwatch. Memory is read straight from PyTorch, and the "peak" counter is reset so that later we can see the most memory used *while answering*.

13.3 Part 2: build the input exactly as the authors do

The model was trained on conversations in a fixed layout (the "vicuna_v1" template), so it must be asked in the same layout. This is the complete text the model receives for VQA-RAD question 0:

```
A chat between a curious user and an artificial intelligence assistant. The assistant gives
helpful, detailed, and polite answers to the user's questions. USER: <image>
is there evidence of an aortic aneurysm? ASSISTANT:
```



Figure 15. What the language model actually reads. The image becomes 576 tokens, about 92% of the input. That is why questions and report prompts cost almost the same to read (TTFT is ≈0.035 s for both).

scripts/bench_inference.py (input)

```
conv = conv_templates["vicuna_v1"].copy()
conv.append_message(conv.roles[0], DEFAULT_IMAGE_TOKEN + "\n" + qs)      # USER: <image>\nquestion
conv.append_message(conv.roles[1], None)                                 # ASSISTANT: (empty, to be filled)
input_ids = tokenizer_image_token(conv.get_prompt(), tokenizer, IMAGE_TOKEN_INDEX, …)
image = Image.open(os.path.join(args.image_folder, q["image"])).convert("RGB")
image_tensor = process_images([image], image_processor, model.config)[0]
images = image_tensor.unsqueeze(0).half().to("cuda:0")                   # fp16, batch of 1
```

`tokenizer_image_token` turns the text into token numbers and leaves a special placeholder (the number −200) where `<image>` was. Inside `generate`, the LLaVA code swaps that placeholder for the 576 image vectors. `process_images` prepares the picture. Because the model's configuration says `"image_aspect_ratio": "pad"`, a non-square image is first padded to a square with a grey colour (the average colour CLIP was trained on), then resized to 336 × 336:

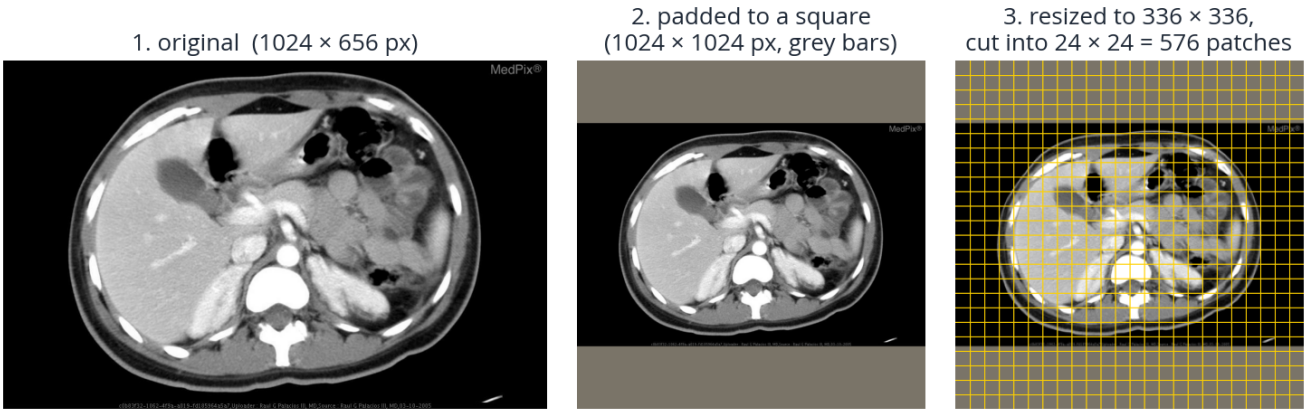


Figure 16. What `process_images` does to VQA-RAD image 0004 (an abdominal CT slice). Padding instead of cropping means nothing at the edges of a medical image is cut off. The yellow grid shows the 24 × 24 patches that become the 576 image tokens.

For reports there is no question, so the script picks one of five instructions ("Walk through the important details of the image", "Analyze the image in a comprehensive and detailed manner", …) at random, exactly as the authors' report script does. `random.seed(42)` makes that choice the same on every run.

13.4 Part 3: generate, with the authors' settings

scripts/bench_inference.py (generation)

```
output_ids = model.generate(input_ids, images=images,
    do_sample=args.temperature > 0, temperature=args.temperature, top_p=None, num_beams=1,
    max_new_tokens=max_new_tokens, use_cache=True,
    pad_token_id=tokenizer.eos_token_id, streamer=streamer)
```

Setting	Value	Plain meaning
`do_sample`, `temperature`	on, 0.2	At each step the model has a probability for every possible next token. With sampling, the next token is drawn at random from those probabilities. Temperature 0.2 sharpens them strongly towards the most likely token, so answers are nearly, but not perfectly, repeatable.
`num_beams`, `top_p`	1, none	No beam search (exploring several candidate answers in parallel) and no extra filtering. One answer is built token by token.
`max_new_tokens`	64 / 1024	Safety cap on answer length: 64 for questions, 1024 for reports. Generation stops earlier if the model emits its end-of-answer token.
`use_cache`	on	Keep the KV cache (Chapter 2), so each new token only needs work for itself, not for the whole prompt again.
batch size	1	One question at a time, as in the authors' scripts. Processing several at once ("batching") can raise throughput, but we did not measure it.
precision	fp16	Weights and image in 16-bit floating point (Chapter 2).

Table 10. Generation settings. All match `MMedPO/inference/llava-med-1.5_vqa.py` and `_report.py`.

13.5 Part 4: the stopwatch, with timestamps at every stage

A plain stopwatch around `generate()` gives the total, but not the parts. To see inside, we used a hook that Hugging Face provides for printing text as it is generated: a **streamer**. `generate()` calls the streamer's `put()` method once with the prompt, just before generation starts, and then once for every new token. Our streamer prints nothing. It only writes down the time of each call:

scripts/bench_inference.py (the streamer)

```
class TimingStreamer(BaseStreamer):
    """generate() calls put() once before the first decode step, then once per new token."""
    def __init__(self):  self.times = []
    def put(self, value): self.times.append(time.perf_counter())
    def end(self): pass
```

end_to_end_s = t_end − t_start: one sample, from opening the image file to having the answer

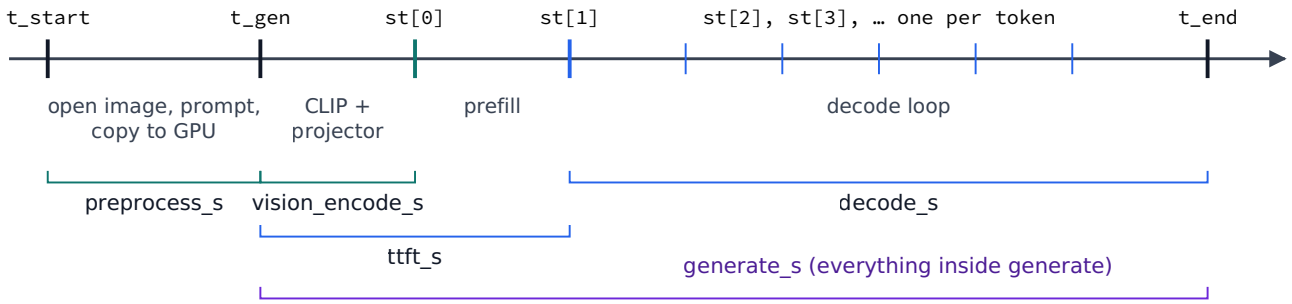


Figure 17. The timestamps recorded for every sample and the durations computed from them. `st[…]` are the times of the streamer's `put()` calls.

Why `torch.cuda.synchronize()` appears before every clock reading

GPUs work **asynchronously**. When Python tells the GPU to do something, it does not wait. It drops the order in a queue and moves on immediately, while the GPU works through the queue on its own. Reading the clock right after a GPU command therefore measures how long it took to *place the order*, not to *cook the meal*. `torch.cuda.synchronize()` means "wait until the GPU has finished everything queued". We call it before every timestamp that matters, so the stopwatch measures real work.

Warm-up

The first samples of a run pay one-time costs (Chapter 7: about 15 s for the very first sample on a fresh system). Each sample is recorded with `"warmup": true` if it is among the first 3. These are left out of every average, median and percentile, so the averages describe steady-state speed. They *are* included in `total_inference_s`, the "whole test set" time, because in real life you would pay them once too.

13.6 Part 5: what it writes

For every sample, one line in *tag*_answers.jsonl. Here is VQA-RAD sample 1 from the full run:


```
{"question_id": 1, "image": "0001.png", "prompt": "is there airspace consolidation on the left
side?",
 "answer": "Yes", "gt_answer": "yes", "answer_type": "CLOSED", "warmup": true, "new_tokens": 4,
 "prompt_tokens": 627, "preprocess_s": 0.0404, "vision_encode_s": 0.0206, "ttft_s": 0.0419,
 "decode_s": 0.0212, "generate_s": 0.0632, "end_to_end_s": 0.1035}
```


"Yes" plus the end-of-answer marker counts as 4 new tokens (word pieces and special tokens). Numbers are shortened to 4 decimals here.

At the end, one *tag*_summary.json with the averages. Chapter 15 explains every field. The script also performs a rough **sanity check** for VQA: for CLOSED (yes/no) questions, it counts an answer as right if the reference word appears among the first three words of the model's answer. This is not the paper's official metric. It exists only to catch a broken setup: if accuracy came out near 0% or 50%, something would be wrong with how we feed the model.

CHAPTER 14

Step 5 · Four runs at once

`scripts/run_bench.sbatch` runs the timing script on four test sets at the same time, each on its own GPU. One job, four GPUs, eleven minutes.

14.1 The script after the header

scripts/run_bench.sbatch (lines 9–33)

```
set -euo pipefail
W=/home/shovan.1/MRI-MMedPO
export HF_HOME=$W/hf_cache HF_HUB_OFFLINE=1 PYTHONUNBUFFERED=1
BASE=$W/checkpoints/llava-med-v1.5-mistral-7b
OUT=$W/results/full
mkdir -p $OUT

nvidia-smi --query-gpu=index,memory.used,utilization.gpu --format=csv
mapfile -t GPUS < <(nvidia-smi --query-gpu=index,memory.used --format=csv,noheader,nounits | awk -F', ' '$2<1000{print $1}' | head -4)
[ ${#GPUS[@]} -ge 4 ] || { echo "need 4 idle GPUs, found ${#GPUS[@]}"; exit 1; }
echo "using GPUs ${GPUS[*]}"

run() {  # gpu tag task lora question_file image_folder
  CUDA_VISIBLE_DEVICES=$1 $W/.venv/bin/python $W/scripts/bench_inference.py --tag $2 --task $3 \
    --model-path $W/checkpoints/llava-med-1.5_$4_mmedpo_lora --model-base $BASE \
    --question-file $5 --image-folder $6 --out-dir $OUT > $W/logs/bench_$2.log 2>&1
}
run ${GPUS[0]} vqa-rad vqa    vqa-rad $W/data/vqa-rad/test.jsonl        $W/data/vqa-rad/images &
run ${GPUS[1]} slake   vqa    slake   $W/data/slake/test.jsonl          $W/data/slake/imgs &
run ${GPUS[2]} iuxray  report iuxray  $W/data/iuxray/test_mmedpo.jsonl  $W/data/iuxray/images &
# MIMIC-CXR needs PhysioNet credentials; time the MIMIC checkpoint on IU-Xray chest X-rays instead.
run ${GPUS[3]} mimic-ckpt_on_iuxray report mimic $W/data/iuxray/test_mmedpo.jsonl $W/data/iuxray/images &
wait
echo BENCH_DONE
```

Line by line, the new ideas:

- `HF_HUB_OFFLINE=1`: no network during the timed run (Chapter 8). `PYTHONUNBUFFERED=1` makes Python write each progress line to the log immediately rather than in batches, so `tail -f` shows progress live.
- **`mapfile -t GPUS < <( … )`** runs the GPU-picking pipeline from Chapter 5, this time keeping up to four idle GPUs (`head -4`), and stores the result as a bash *array* called `GPUS`. `${GPUS[0]}` is the first element, `${#GPUS[@]}` the number of elements.
- The `[ … ] || { …; exit 1; }` line is a guard: if fewer than four GPUs are idle, stop with a clear message instead of starting runs that would collide with someone else.
- **`run() { … }`** defines a shell *function*, a named mini-script. Inside it, `$1` … `$6` are its arguments, in the order given in the comment. Writing `CUDA_VISIBLE_DEVICES=$1 command` sets the variable *for that one command only*, so each run sees just its own GPU as `cuda:0`.

- `> file 2>&1` sends the program's normal output (`>`) and its error output (`2>&1` , "stream 2 goes wherever stream 1 goes") into one log file per run: `logs/bench_vqa-rad.log` and so on.
- **The `&` at the end of each `run` line** starts it in the *background* and moves straight on to the next line. All four runs start within a fraction of a second. **`wait`** then pauses the script until every background run has finished. Without `wait` , the script would reach its end immediately, Slurm would consider the job done, and it would kill the four runs.

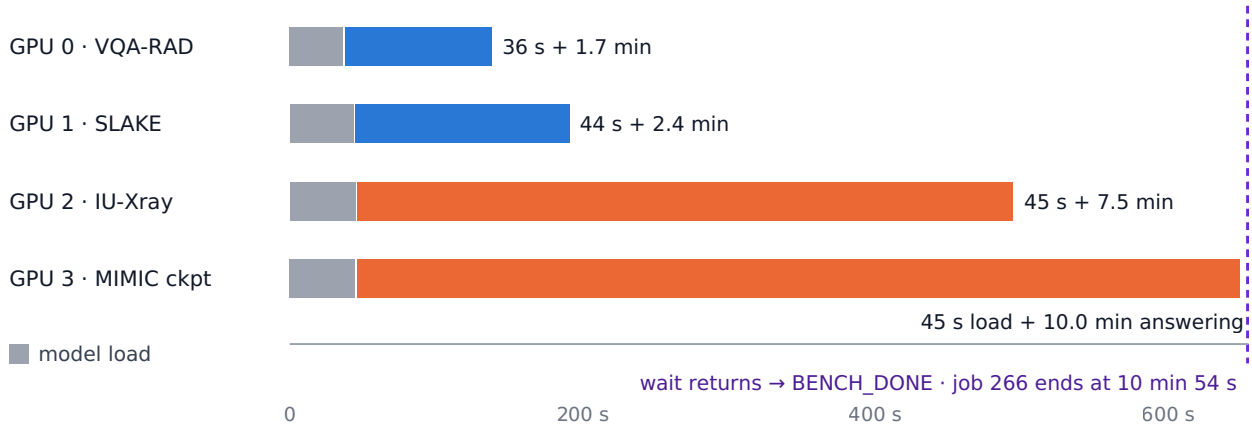


Figure 18. Job 266 on a time axis, drawn to scale from the logged numbers. Each lane is one GPU. Grey is loading (36–45 s, slower than alone because four loads read the network disk at once); coloured is answering every test question. The job ends when the slowest run finishes.

14.2 What you see while it runs

```
$ grep '^\[' logs/bench_vqa-rad.log
[1/451] 15.151s 4 tok | 'No'
[2/451] 0.063s 4 tok | 'Yes'
[3/451] 0.068s 4 tok | 'No'
[100/451] 0.057s 4 tok | 'Yes'
[200/451] 0.059s 4 tok | 'No'
[300/451] 0.356s 31 tok | 'The mass is quite large, but the exact dimensions can vary depending on
the size of the mo'
[400/451] 0.196s 17 tok | 'In the lung apices, there are bilateral nodules.'
```

The job's own log (`logs/mmedpo-bench-266.out`) holds only the GPU table, "using GPUs 0 1 2 3" and `BENCH_DONE` . Each run's log shows the first three samples and every 100th, with time, token count and the start of the answer, so you can watch progress with `tail -f logs/bench_iuxray.log` .

> **WHY FOUR GPUS IN ONE JOB, RATHER THAN ONE GPU FOR LONGER?**
> The four test sets are completely independent, so running them side by side cut the total from roughly 23 minutes (the four runs back to back) to 11. This is the simplest kind of parallelism, sometimes called "embarrassingly parallel": different data, same program, no communication. It is also how you would scale to a big hospital archive: split the images into eight files and run eight copies on eight GPUs.

CHAPTER 15

Step 6 · Collecting the numbers

Each run leaves a small JSON summary. `scripts/summarize.py` reads all of them and prints one table. It reads a few small files, so it runs on the login node in an instant.

```
$ .venv/bin/python scripts/summarize.py
| Run | Task | Samples | Load (s) | GPU mem peak (GB) | Per-sample mean (s) | Median (s) | p90 (s)
| … |
|---|---|---|---|---|---|---|---|---|
| iuxray | report | 590 | 44.9 | 16.6 | 0.734 | 0.645 | 0.908 | … |
| mimic-ckpt_on_iuxray | report | 590 | 44.5 | 16.8 | 0.997 | 0.921 | 1.285 | … |
| slake | vqa | 1061 | 43.5 | 16.6 | 0.124 | 0.063 | 0.146 | … |
| vqa-rad | vqa | 451 | 36.4 | 16.6 | 0.188 | 0.065 | 0.284 | … |
slake: closed-ended accuracy (rough check) = 73.6%
vqa-rad: closed-ended accuracy (rough check) = 64.9%
```

The table is in Markdown, a plain-text table format that pastes neatly into reports. Part IV shows it in full. Here is what every field of a summary file means:

Field	Meaning	IU-Xray value
`n_samples_total` / `_timed`	Samples answered / samples used for averages (total minus 3 warm-up)	590 / 587
`model_load_s`	Load + merge time (Chapter 12)	44.85
`weights_gpu_mem_gb`	GPU memory right after loading	16.28
`peak_gpu_mem_gb`	Most GPU memory in use at any moment while answering	16.56
`generate_s_mean` / `_median` / `_p90` / `_max`	Time inside `generate()`: average, middle value, 90th percentile (90% of samples were faster), slowest	0.700 / 0.645 / 0.908 / 1.489
`end_to_end_s_mean`	Average per-sample time including opening the image and building the prompt	0.734
`ttft_s_mean`	Average time to the first answer token	0.035
`vision_encode_s_mean`	Average CLIP + projector time	0.018
`new_tokens_mean` / `_max`	Answer length in tokens	63.7 / 138
`decode_tok_per_s_mean`	Tokens per second after the first one	94.4
`total_inference_s`	Sum of every sample's time, warm-up included: the whole test set	447.4 (7.5 min)
`throughput_samples_per_s`	Timed samples ÷ their total time	1.36
`sanity_closed_acc`	VQA only: rough yes/no accuracy (Chapter 13)	n/a

Table 11. The fields of `results/full/iuxray_summary.json`.

Part IV

Results

What the measurements say, why they look the way they do, what they let you estimate, and where their limits are.

CHAPTER 16

What we measured

The answer to the advisor, in one table: on one B200, MMedPO answers a medical question in about a tenth of a second and writes a chest X-ray report in under a second, using under 9% of the GPU's memory.

Run (checkpoint → test set)	Samples	Mean per sample (s)	Median generate (s)	p90 (s)	Max (s)	Tokens out (mean)	Decode tok/s	Peak GPU mem (GB)	Whole set (min)
VQA-RAD → VQA-RAD	451	0.188	0.065	0.284	0.74	11	118	16.6	1.7
SLAKE → SLAKE	1061	0.124	0.063	0.146	0.72	7	124	16.6	2.4
IU-Xray → IU-Xray reports	590	0.734	0.645	0.908	1.49	64	94	16.6	7.5
MIMIC → IU-Xray reports*	590	0.997	0.921	1.285	11.13	92	108	16.8	10.0

Table 12. Full results, job 266 (28 Oct 2026), one NVIDIA B200 per run, batch size 1, fp16. "Mean per sample" is end-to-end, including image loading. Median, p90 and max are of `generate()` time. "Whole set" includes the warm-up samples. *Stand-in for MIMIC-CXR (Chapter 8). TTFT was 0.034–0.036 s in every run.

MEAN OR MEDIAN?

For VQA, the mean (0.088 s for SLAKE) is well above the median (0.063 s), because a minority of open questions get long answers that pull the average up. The median tells you what a "typical" question costs; the mean tells you how long a whole batch takes (mean × count). Report both, as we did.

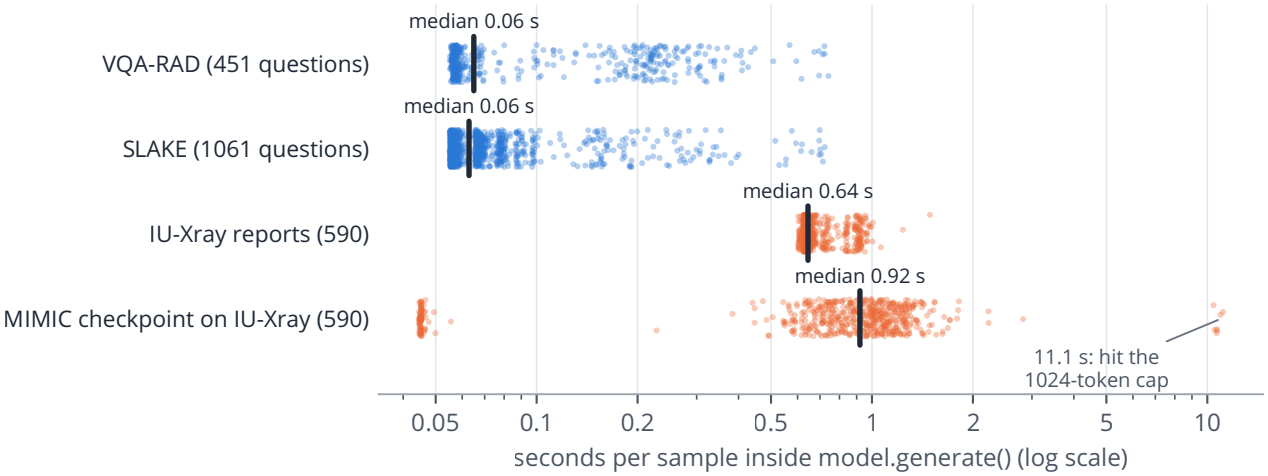


Figure 19. Every timed sample (one dot each, warm-up excluded) on a log scale. Blue are question-answering runs, orange report runs; the black bar is the median. The isolated points far right are the 1024-token runaway reports discussed below. The cluster near 0.05 s in the bottom row is empty answers.

16.1 Answer length explains almost everything

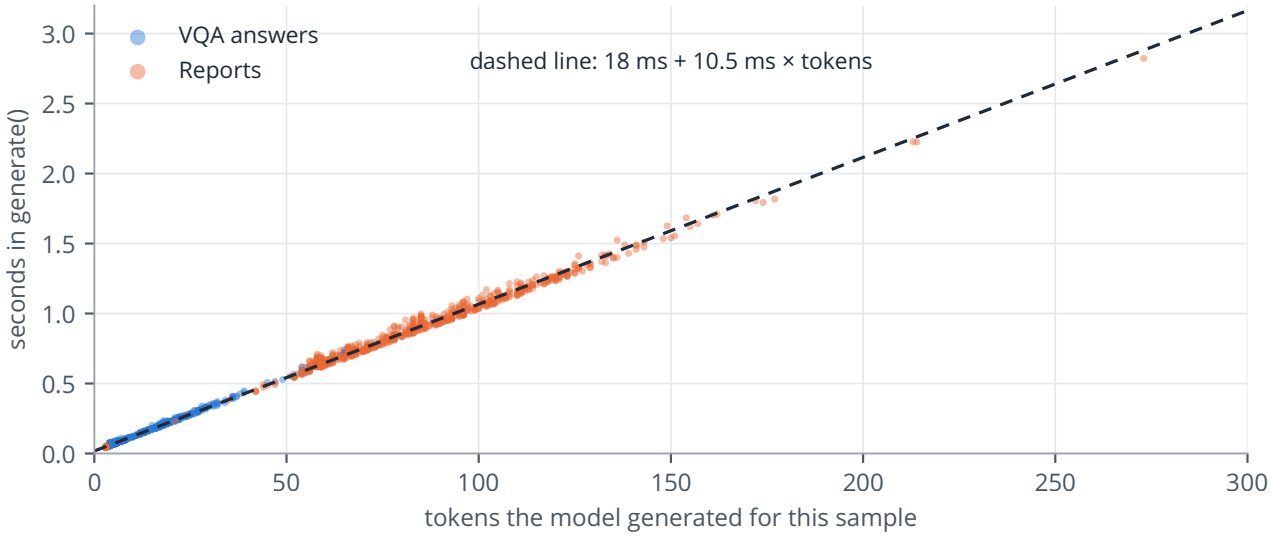


Figure 20. Time inside `generate()` against the number of tokens generated, for all 2,673 timed samples with at most 300 output tokens. They fall on a straight line: about 18 ms of fixed cost (image encoding and reading the prompt) plus 10.5 ms per generated token.

This is Chapter 1's "reading versus writing" picture in real data. Reading the prompt, including 576 image tokens, is a fixed ~18–35 ms. Every additional word costs about 10.5 ms, roughly 95 tokens per second. A one-word answer ("Yes") therefore takes ~0.06 s, and a 64-token report ~0.7 s. The fitted line also predicts the outliers: 1024 tokens × 10.5 ms ≈ 10.8 s, matching the 11.1 s maximum.

It also explains the differences between the runs. VQA-RAD is slower on average than SLAKE because its open questions draw longer answers (11 vs 7 tokens on average). The MIMIC checkpoint is slower than the IU-Xray checkpoint on the same images because it writes longer reports (92 vs 64 tokens).

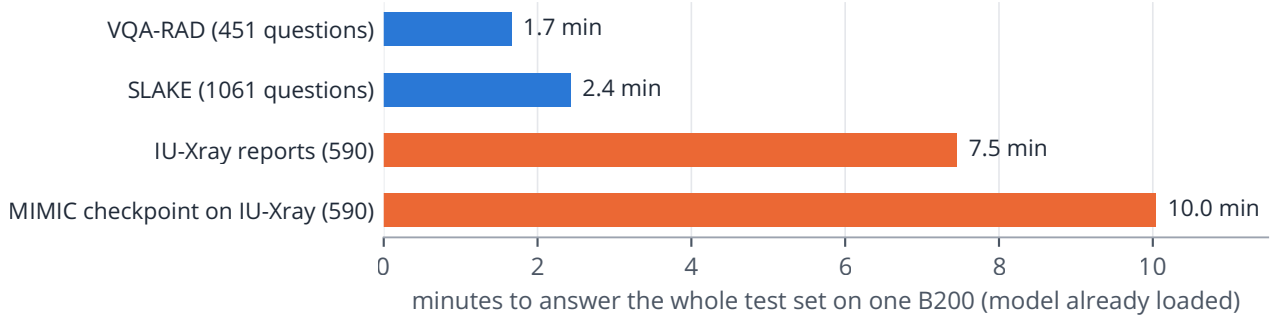


Figure 21. Time to answer each complete test set on one GPU, warm-up included, model load excluded (add 36–45 s).

16.2 Where the time goes in one request

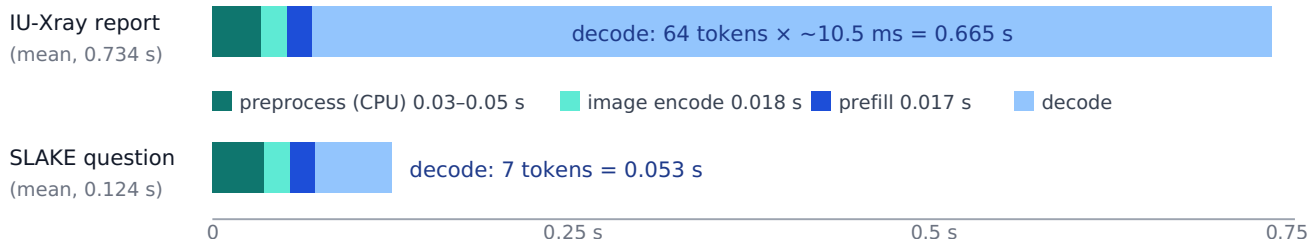


Figure 22. Average breakdown of one request, drawn to scale from the per-sample timestamps. For reports, decoding is 90% of the time. For short answers, the fixed costs dominate, including about 30% spent on the CPU opening and resizing the image.

16.3 Things the benchmark caught along the way

The first sample: 15 seconds

In all four runs the first sample took 15.05–15.15 s, then everything was fast. Chapter 7 explained why (one-time GPU initialisation and compilation, cached afterwards), and Chapter 13 why it does not distort the averages.

Runaway reports: the 1024-token cap

For 7 of the 590 images, the MIMIC checkpoint never produced its end-of-answer token. It fell into a loop and repeated a sentence until it hit the 1024-token safety cap, at about 11 s each. This is the end of one of them, from `results/full/mimic-ckpt_on_iuxray_answers.jsonl`:

```
… The right lung is hyperinflated. The right lung is hyperinflated. The right lung is
hyperinflated. The right lung is hyperinflated. The right lung is hyperinflated
```

Repetition loops are a known failure of language models. The cap is what keeps them from running for minutes: always set `max_new_tokens`.

Empty reports from the MIMIC checkpoint

77 of the MIMIC checkpoint's 590 reports were empty: the model emitted its end token almost immediately (the dots near 0.05 s in the distribution figure). 73 of them came from one of the five report instructions, "Examine the image closely and share its details", which produced an empty report 73 times out of the 110 times it was drawn. The IU-Xray checkpoint never produced an empty report with any instruction. This does not affect the timing question, but it suggests the MIMIC checkpoint is sensitive to prompt wording on images from outside its training data. It is worth remembering if MIMIC-CXR is ever evaluated properly.

Is the setup actually right? The accuracy check

Timing a model that is fed incorrectly would be meaningless, so the rough yes/no check matters. It came out close to the paper's official closed-question accuracies:

Test set	Our rough check	Paper, Table 1 (MMedPO)
SLAKE (closed)	73.6 %	73.08 %
VQA-RAD (closed)	64.9 %	66.54 %

Table 13. The closeness says the model, adapters, prompts and preprocessing are wired up correctly. It is not a replication of the paper's evaluation: the metric is our own simple version, with newer software and sampling at temperature 0.2.

16.4 Back-of-the-envelope: what would a bigger job cost?

The per-sample averages let you estimate larger workloads. Take the slower run of each kind (VQA-RAD for questions, the MIMIC checkpoint for reports), assume free GPUs, and simply divide the work between GPUs:

Workload	1,000 items	10,000 items	100,000 items
Questions (0.188 s each), 1 GPU	3.1 min	31 min	5.2 h
Questions, 8 GPUs	0.4 min	3.9 min	39 min
Reports (0.997 s each), 1 GPU	17 min	2.8 h	27.7 h ✗
Reports, 8 GPUs	2.1 min	21 min	3.5 h

Table 14. Estimates, not measurements. Add about a minute per GPU for loading. ✗ = longer than the 6-hour job limit, so it must be split into several jobs (for example, one question file per job).

Two things could make real large runs faster than this: batching several images per `generate()` call, and spreading CPU preprocessing over more cores. We measured neither. These estimates are conservative.

> **RECAP**
>
> - Questions: ~0.06 s typical, 0.12–0.19 s average. Reports: 0.7–1.0 s average. Memory: 16.6–16.8 GB peak.
> - Time ≈ 18 ms + 10.5 ms × output tokens, so answer length drives the cost.
> - One-time costs: 5–45 s to load and ~15 s for the very first sample on a cold system.
> - The accuracy sanity check matches the paper, so the setup is sound.

CHAPTER 17

What these numbers do and do not tell you

Every measurement has a scope. Here is exactly what ours covers, so that nobody reads more into it than it says.

Caveat	What it means for the numbers
Newer software than the paper	torch 2.8.0 instead of 2.1.2, plus matching newer transformers/peft/accelerate. Unavoidable: 2.1.2 cannot run on a B200 at all (job 275). Answers may differ slightly from the authors' runs; timing on our hardware is what it is.
MIMIC-CXR not tested	No PhysioNet access. The MIMIC checkpoint was timed on IU-Xray chest X-rays instead. Same model and task type, different images. MIMIC reports are often longer, so real MIMIC timings could be somewhat higher.
Inference only	Training (data curation with GPT-4o, MedKLIP and Med-LLM scoring, then DPO) was not run or timed. See Chapter 20.
One GPU per model, batch size 1	We did not split a model across GPUs (unnecessary at 17 GB) or batch requests. Throughput numbers are for the simplest setup and are likely conservative.
No GPU accounting by Slurm	We ran when all GPUs were idle (0 MiB in use). On a busy day, sharing a GPU with someone else would slow things down, and Slurm would not prevent it.
Four runs shared one node	Loading took longer in parallel (36–45 s vs 17 s alone). CPU-side steps may have been slightly slower too. GPU work was not shared.
Sampling (temperature 0.2)	Answers, and so answer lengths and times, vary a little between runs. Averages over hundreds of samples are stable; single samples are not.
Rough accuracy only	We did not compute the paper's metrics (open-question recall, BLEU, ROUGE-L, METEOR). The yes/no check is a wiring test, not a replication.
Images were already PNG/JPG	Hospital images arrive as DICOM files. Converting them would add CPU time that we did not measure.

Table 15. Scope of the measurements.

WHAT YOU CAN TELL YOUR ADVISOR

"Yes, MMedPO runs on the cluster. It needs about 17 GB of a B200's 183,359 MiB, so memory is not an issue. With the released checkpoints on one B200 it answers a VQA question in 0.12–0.19 s on average (about 0.06 s typical) and writes a chest X-ray report in 0.7–1.0 s; the full test sets take 1.7–10 minutes each. The only obstacle was that the paper's PyTorch 2.1.2 cannot run on B200 GPUs, so we used PyTorch 2.8. Our yes/no accuracy check matches the paper (73.6% vs 73.08% on SLAKE). MIMIC-CXR itself was not tested because it needs PhysioNet credentials; training was not measured."

Part V

Do it yourself

A hands-on lab you can type along with, a troubleshooting guide for what can go wrong, and a sketch of what it would take to go beyond inference and actually train MMedPO.

CHAPTER 18

Hands-on lab

In about fifteen minutes you will get a GPU, run the model on ten real questions, ask it your own question about an X-ray, have it write a report, and then do the same thing as a batch job. Every command below was run exactly like this on 29 October 2026 (jobs 269, 271 and 274), and every output shown is the real one.

BEFORE YOU START

- Open a terminal on the cluster: the Claude Code app's terminal, or `ssh` from your computer. You start on the login node.
- Everything the lab writes goes into `~/MRI-MMedPO/results/lab/`. Nothing in it touches the benchmark results.
- The lab uses one GPU for a few minutes. Check that it is not busy (step 3) and type `exit` when you are done.

STEP 1 · WHERE AM I, AND IS THE CLUSTER FREE?

```
$ hostname
aihub-cluster1
$ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
hgx*         up    6:00:00      1   idle hgx-01
$ squeue
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
```

You are on the login node. hgx-01 is `idle` (nobody is using it) and the queue is empty. If it said `mix`, some of it would be in use, which is fine; `alloc` means full, and your job would wait.

STEP 2 · GET AN INTERACTIVE SHELL ON HGX-01

```
[shovan.1@aihub-cluster1 ~]$ srun -p hgx -c 8 -t 01:00:00 --pty bash
[shovan.1@hgx-01 ~]$ hostname
hgx-01.aihub.lan
```

8 CPUs (so 15.2 GB of RAM) for up to one hour. The prompt now says `hgx-01`: every command you type from here runs on the GPU machine. (Your prompt may look slightly different, but it will show the machine name.)

STEP 3 · LOOK AT THE GPUS AND PICK A FREE ONE

```
$ nvidia-smi --query-gpu=index,name,memory.used,memory.total,utilization.gpu --format=csv
index, name, memory.used [MiB], memory.total [MiB], utilization.gpu [%]
0, NVIDIA B200, 0 MiB, 183359 MiB, 0 %
1, NVIDIA B200, 0 MiB, 183359 MiB, 0 %
2, NVIDIA B200, 0 MiB, 183359 MiB, 0 %
3, NVIDIA B200, 0 MiB, 183359 MiB, 0 %
4, NVIDIA B200, 0 MiB, 183359 MiB, 0 %
5, NVIDIA B200, 0 MiB, 183359 MiB, 0 %
6, NVIDIA B200, 0 MiB, 183359 MiB, 0 %
7, NVIDIA B200, 0 MiB, 183359 MiB, 0 %
$ export CUDA_VISIBLE_DEVICES=$(nvidia-smi --query-gpu=index,memory.used --
format=csv,noheader,nounits | awk -F', ' '$2<1000{print $1; exit}')
$ echo "Using GPU $CUDA_VISIBLE_DEVICES"
Using GPU 0
```

STEP 4 · ENTER THE PROJECT AND SWITCH ON ITS PYTHON

```
$ cd ~/MRI-MMedPO
$ source .venv/bin/activate
$ export HF_HOME=$PWD/hf_cache HF_HUB_OFFLINE=1
$ which python
/home/shovan.1/MRI-MMedPO/.venv/bin/python
$ python -c "import torch; print(torch.__version__, torch.cuda.is_available(),
torch.cuda.get_device_name(0))"
2.8.0+cu128 True NVIDIA B200
```

`$PWD` is “the folder I am in”. The last line is the most useful 5-second check in GPU work: the right PyTorch, a GPU is visible, and it is the B200 you expect.

STEP 5 · TEN REAL QUESTIONS

```
$ python scripts/bench_inference.py --task vqa --tag lab-vqa-rad \
    --model-path checkpoints/llava-med-1.5_vqa-rad_mmedpo_lora \
    --model-base checkpoints/llava-med-v1.5-mistral-7b \
    --question-file data/vqa-rad/test.jsonl --image-folder data/vqa-rad/images \
    --out-dir results/lab --limit 10
Loading LLaVA from base model...
Loading checkpoint shards: 100%|██████████| 4/4 [00:03<00:00, …]
Loading additional LLaVA weights...
Loading LoRA weights...
Merging LoRA weights...
Model is loaded...
[1/10] 1.452s 4 tok | 'No'
[2/10] 0.060s 4 tok | 'Yes'
[3/10] 0.060s 4 tok | 'No'
{
  "tag": "lab-vqa-rad",
  …
  "model_load_s": 5.453926986083388,
  "peak_gpu_mem_gb": 16.562806272,
  "generate_s_mean": 0.16899803819667017,
  "ttft_s_mean": 0.038182332180440426,
  "decode_tok_per_s_mean": 114.05803588216273,
```

```
  "sanity_closed_acc": 0.0
}
```

The model loaded in 5.5 s, because the files were still cached in memory from earlier runs. The first answer took 1.45 s: the warm-up again, much shorter now that the GPU code cache is filled. After that, 0.06 s per yes/no answer. Now look at what it actually said. The answers are in `results/lab/lab-vqa-rad_answers.jsonl`:

#	Question	Reference	MMedPO said
1*	is there evidence of an aortic aneurysm?	yes	No
2*	is there airspace consolidation on the left side?	yes	Yes
3*	is there any intraparenchymal abnormalities in the lung fields?	no	No
4	which side of the heart border is obscured?	right	The right side of the heart border is obscured in the image.
5	where are the kidney?	not seen here	The kidney is located in the right upper quadrant of the abdomen.
6	are the kidneys present in this image?	no	Yes
7	is the colon more prominent on the patient's right or left side?	left	The colon appears to be more prominent on the patient's right side.
8	where is the colon most prominent from this view?	left	The colon is most prominent from the right side in this view.
9	is the heart size in this image smaller or larger than if the image was taken ap?	smaller	The heart size in this image is smaller than if the image was taken ap
10	is the liver visible in the image?	no	Yes

Table 16. The ten lab answers. * = warm-up sample, not included in averages. Green matches the reference, red does not.

Two lessons. First, `sanity_closed_acc: 0.0` is not a bug. Among the seven timed samples only two were yes/no questions (6 and 10), and the model got both wrong. Ten samples say nothing about accuracy; the full run's 64.9% does. Second, look at *how* it is wrong: it claims organs are visible that are not, and confuses left and right. These are exactly the kinds of hallucinations the MMedPO paper tries to reduce, and they are why a model like this is a research tool, not a diagnostic one.

STEP 6 · ASK YOUR OWN QUESTION

Any image works. Put it in a folder, write a one-line question file, and point the script at both. Here we use a chest X-ray from IU-Xray, saved as `chest.png` as if it were your own file, and the SLAKE checkpoint. `--warmup 0` makes the single sample count.

```
$ mkdir -p results/lab/my_images
$ cp data/iuxray/images/CXR3030_IM-1405/0.png results/lab/my_images/chest.png
$ echo '{"question_id": 0, "image": "chest.png", "question": "Is the heart size normal? Explain
briefly."}' > results/lab/my_question.jsonl
$ python scripts/bench_inference.py --task vqa --tag my-question \
    --model-path checkpoints/llava-med-1.5_slake_mmedpo_lora \
    --model-base checkpoints/llava-med-v1.5-mistral-7b \
    --question-file results/lab/my_question.jsonl --image-folder results/lab/my_images \
    --out-dir results/lab --limit 1 --warmup 0
…
Model is loaded...
[1/1] 0.517s 4 tok | 'No'
$ python -c "import json; print(json.loads(open('results/lab/my-
question_answers.jsonl').readline())['answer'])"
No
```

CHECK THE ANSWER AGAINST THE TRUTH

The radiologist's report for this X-ray begins "Normal cardiomediastinal silhouette", which means the heart's outline is normal. So the model's "No" is **wrong**, and it ignored "Explain briefly": the SLAKE checkpoint was tuned on short answers. A wrong answer from a single question is not a verdict on the model, but it is a good habit to check outputs against a reference whenever one exists.

STEP 7 · ASK FOR A REPORT

For reports the question file needs only the image; the script supplies the instruction. Use the IU-Xray checkpoint:

```
$ echo '{"question_id": 0, "image": "chest.png"}' > results/lab/my_report.jsonl
$ python scripts/bench_inference.py --task report --tag my-report \
    --model-path checkpoints/llava-med-1.5_iuxray_mmedpo_lora \
    --model-base checkpoints/llava-med-v1.5-mistral-7b \
    --question-file results/lab/my_report.jsonl --image-folder results/lab/my_images \
    --out-dir results/lab --limit 1 --warmup 0
…
[1/1] 1.301s 85 tok | 'The chest x-ray shows a normal heart size and mediastinal and hilar
contours. The pulmonar'
```

The full report, from `results/lab/my-report_answers.jsonl`, next to the real one:

MMEDPO (IU-XRAY CHECKPOINT)
The chest x-ray shows a normal heart size and mediastinal and hilar contours. The pulmonary vasculature is normal, and the lungs are clear. No pleural effusion or pneumothorax is present, and there is no evidence of acute osseous abnormalities. The impression is that there is no acute cardiopulmonary abnormality.

RADIOLOGIST (GROUND TRUTH)
Normal cardiomediastinal silhouette. There is no focal consolidation. There are no XXXX of a large pleural effusion. There is no pneumothorax. There is no acute bony abnormality seen.

This time the model agrees with the radiologist: normal heart, no fluid, no collapsed lung, no bone injury. It used different words for the same findings ("osseous" means bone). Notice that two checkpoints, asked about the same image, gave contradictory answers about the heart. They are differently tuned models.

STEP 8 · LEAVE THE GPU

```
[shovan.1@hgx-01 MRI-MMedPO]$ exit
[shovan.1@aihub-cluster1 ~]$
```

This ends the job and frees the GPU. If you forget, the job ends by itself when its hour runs out.

STEP 9 · THE SAME THING AS A BATCH JOB

Interactive sessions are great for poking around. For anything you want to leave running, write a batch script. This one runs 20 SLAKE questions; it is in `results/lab/my_job.sbatch`:

results/lab/my_job.sbatch

```
#!/bin/bash
#SBATCH -J my-first-job
#SBATCH -p hgx
#SBATCH -c 8
#SBATCH -t 00:20:00
#SBATCH -o /home/shovan.1/MRI-MMedPO/results/lab/%x-%j.out
set -euo pipefail
cd /home/shovan.1/MRI-MMedPO
export HF_HOME=$PWD/hf_cache HF_HUB_OFFLINE=1
export CUDA_VISIBLE_DEVICES=$(nvidia-smi --query-gpu=index,memory.used --format=csv,noheader,nounits | awk -
F', ' '$2<1000{print $1; exit}')
[ -n "$CUDA_VISIBLE_DEVICES" ] || { echo "no idle GPU right now"; exit 1; }
echo "job $SLURM_JOB_ID on $(hostname), using GPU $CUDA_VISIBLE_DEVICES"
nvidia-smi --query-gpu=index,name,memory.used,memory.total --format=csv
.venv/bin/python scripts/bench_inference.py --task vqa --tag batch-slake \
  --model-path checkpoints/llava-med-1.5_slake_mmedpo_lora \
  --model-base checkpoints/llava-med-v1.5-mistral-7b \
  --question-file data/slake/test.jsonl --image-folder data/slake/imgs \
  --out-dir results/lab --limit 20
```

```
$ sbatch results/lab/my_job.sbatch
Submitted batch job 274
$ squeue -u $USER
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
               274       hgx my-first shovan.1  R       0:02      1 hgx-01
$ sacct -j 274 -X --format=JobID,JobName%15,State,Elapsed,AllocCPUS,ReqMem,ExitCode
JobID                JobName      State    Elapsed  AllocCPUS     ReqMem ExitCode
------------ --------------- ---------- ---------- ---------- ---------- --------
274             my-first-job  COMPLETED   00:00:13          8     15200M      0:0
$ grep -E '^(job|\[|Model)' results/lab/my-first-job-274.out
job 274 on hgx-01.aihub.lan, using GPU 0
Model is loaded...
[1/20] 0.464s 5 tok | 'CT scan'
[2/20] 0.070s 5 tok | 'Chest'
[3/20] 0.069s 5 tok | 'Lung'
```

Thirteen seconds from start to finish, with all 20 answers in `results/lab/batch-slake_answers.jsonl`. You can close your laptop while a batch job runs; it does not care.

THIS EXACT SCRIPT FAILED THE FIRST TIME

Our first version, job 272, printed the GPU table with `nvidia-smi | head -4`. The job died right after that line, with state FAILED. With `set -o pipefail`, a pipeline counts as failed if *any* command in it fails. When `head` has read its 4 lines it exits, and `nvidia-smi`, still writing, can be killed by the broken pipe. That "failure" made `set -e` stop the script. Replacing the line with the `--query-gpu … --format=csv` form fixed it (job 274). Chapter 19 lists this among the common errors.

TRY IT · THINGS TO CHANGE NEXT

- Drop `--limit` to run a whole test set, and compare your averages with the results table in Chapter 16.
- Use your own image and question. Any PNG or JPG works; it does not have to be from these datasets.
- Ask the same question with different checkpoints (slake, vqa-rad, iuxray, mimic) and compare.
- Run it twice: with temperature 0.2, answers occasionally differ. Add `--temperature 0` to switch sampling off (greedy decoding, which is almost always repeatable).

CHAPTER 19

When things go wrong

Errors on a GPU cluster look intimidating but fall into a few families. Three of the rows below happened to us in this project; the rest are the ones you are most likely to meet next.

You see	What it means	What to do
`CUDA out of memory`	The GPU is full. Usually someone else is using the same GPU, because Slurm does not separate GPUs here (Chapter 5).	Run `nvidia-smi`, pick a GPU with ~0 MiB used, set `CUDA_VISIBLE_DEVICES` to it, start again.
`no kernel image is available for execution on the device`	The PyTorch you are using has no code for this GPU (job 275 with torch 2.1.2).	Use the project venv (torch 2.8.0+cu128). Check with `python -c "import torch; print(torch.__version__)"`.
Job stuck in `PD`; reason `(Resources)` or `(Priority)`	hgx-01 is full, or other jobs are ahead of yours. `(None)` right after submitting is normal.	Wait, or ask for fewer CPUs or less time. `squeue` shows the reason.
Job stuck in `PD`; reason mentions a time limit	You asked for more than the 6-hour maximum.	`scancel` it and resubmit with `-t 06:00:00` or less. Split long work into several jobs.
State `TIMEOUT`, log ends with "CANCELLED … DUE TO TIME LIMIT"	The job ran out of its `-t`.	Ask for more time (up to 6 h), or process the data in chunks.
`execve(): /tmp/…: No such file or directory`	The file exists on the login node but not on hgx-01. `/tmp` is not shared (job 270).	Keep scripts and data under `/home`.
Job `FAILED` right after a line with `| head`	`set -o pipefail` treats the broken pipe as a failure (job 272).	Avoid `| head` in `set -euo pipefail` scripts, or end the line with `|| true`.
`ModuleNotFoundError: No module named 'torch'`	You are running the system Python, not the venv.	`source .venv/bin/activate`, or call `.venv/bin/python` directly. `which python` tells you which one you have.
`ModuleNotFoundError: No module named 'llava'`	A script of your own cannot find the authors' code.	Add `MMedPO/train/dpo` to `sys.path`, as `bench_inference.py` does.
Hugging Face `401` / "gated repo" / `LocalEntryNotFoundError`	A repository needs a login or licence acceptance, or you are offline (`HF_HUB_OFFLINE=1`) and the file was never downloaded.	Log in with a Hugging Face token and accept the licence on the website; or run the download step first, without offline mode.
Log shows no "Merging LoRA weights…"	The adapter was not applied, and you are running something other than MMedPO.	Make sure `--model-path` is a `_lora` shortcut and `--model-base` is set (Chapter 12).
Loading takes a minute or more	Several jobs are reading large files from the network disk at once.	Normal (we saw 5–45 s). It happens once per run.

You see	What it means	What to do
Interactive session vanished	Your connection dropped (laptop slept, Wi-Fi changed), which ends `srun --pty` jobs.	Use `sbatch` for anything longer than a few minutes.
Disk filling up	Models and caches are large.	`du -sh ~/MRI-MMedPO/*`. The 5.2 GB `.uv_cache` can be deleted safely.

Table 17. Common problems and fixes. Jobs 270, 272 and 275 are real failures from this project.

THE FOUR QUESTIONS THAT SOLVE MOST PROBLEMS

1. **Where am I?** `hostname` : login node or hgx-01?
2. **Which Python?** `which python` : the project venv?
3. **Which GPU, and is it free?** `echo $CUDA_VISIBLE_DEVICES` and `nvidia-smi`.
4. **What did the job say?** The log file (`-o`), and `sacct -j JOBID` for its final state.

CHAPTER 20

What training would take

We only ran inference, as asked. If the project later needs to *train* MMedPO, for example on new data or with a different base model, here is what that involves. **Nothing in this chapter was run or measured.** It is a reading of the paper and the authors' code, so treat the numbers as rough.

20.1 The three stages

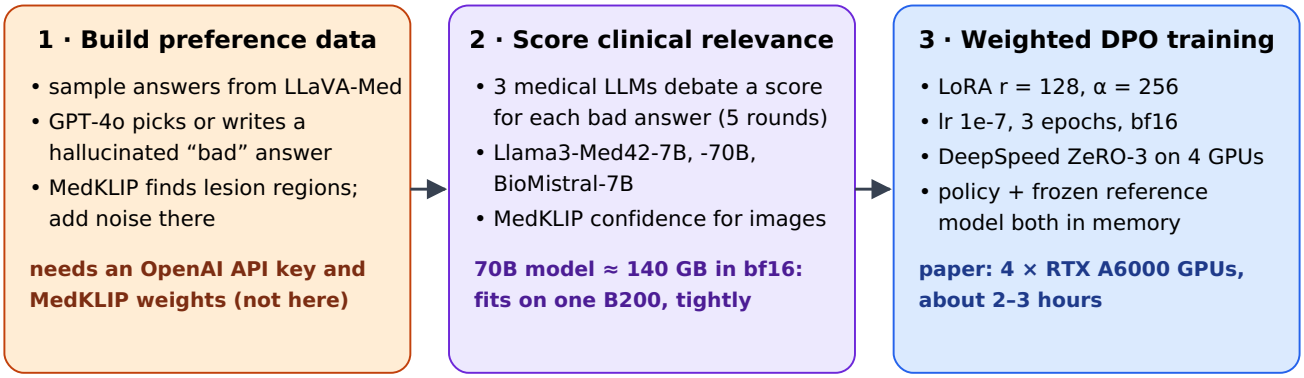


Figure 23. What training MMedPO involves, from Section 3 and Appendix B of the paper and the scripts in `MMedPO/scripts/`. None of it was run.

Ingredient	Status on our cluster
GPU memory	Not a concern. Four B200s far exceed the paper's four 48 GB A6000s. Even the 70B scoring model fits on one B200 (about 140 GB of weights in bf16, out of 183,359 MiB), though with little room to spare.
Software	**The main risk.** The training code relies on DeepSpeed and on DPO-trainer code written for torch 2.1-era libraries. It would need updating to versions that support torch ≥ 2.7 on Blackwell, which we have not tried.
GPT-4o	Needs an OpenAI API key and a budget. Thousands of calls (the paper's training sets range from 1,600 to 9,838 preference pairs).
MedKLIP	Its model file must be obtained separately. The heat-map script is in `MMedPO/curation/`.
Data	Training splits of the four datasets. MIMIC-CXR still needs PhysioNet access.
Time limit	The paper's 2–3 hours fits inside a 6-hour job. On B200s it would probably be faster, but that is unmeasured.

Table 18. Training feasibility at a glance. Unmeasured; the main open question is the software stack, not the hardware.

Appendix A · Command cheat sheet

Slurm

`sinfo`	Is hgx-01 idle, mixed or full?
`squeue -u $USER`	My jobs and their states (PD, R).
`srun -p hgx -c 8 -t 01:00:00 --pty bash`	Interactive shell on hgx-01 for an hour; `exit` to leave.
`srun -p hgx -c 4 -t 00:10:00 COMMAND`	Run one command on hgx-01 and wait for it.
`sbatch FILE.sbatch`	Submit a batch script; prints the job number.
`tail -f LOGFILE`	Follow a log as it grows (Ctrl-C stops watching).
`scancel JOBID`	Stop a job.
`sacct -j JOBID -X`	What happened to a finished job (add `--format=…` for more columns).
`scontrol show partition hgx`	The partition's limits.

GPU and environment (on hgx-01)

`nvidia-smi`	Full GPU status table.
`nvidia-smi --query-gpu=index,memory.used,utilization.gpu --format=csv`	Compact: which GPUs are busy.
`export CUDA_VISIBLE_DEVICES=N`	Use only GPU N (it becomes `cuda:0`).
`cd ~/MRI-MMedPO && source .venv/bin/activate`	Enter the project and switch on its Python.
`export HF_HOME=$PWD/hf_cache HF_HUB_OFFLINE=1`	Use the project's model cache, offline.
`python -c "import torch; print(torch.__version__, torch.cuda.is_available())"`	Five-second sanity check.

This project

`sbatch scripts/setup_env.sbatch`	Rebuild the environment and downloads (safe to re-run).
`sbatch scripts/run_bench.sbatch`	Full benchmark on 4 idle GPUs (~11 min).
`python scripts/bench_inference.py --task vqa|report … --limit N`	Time one checkpoint on one question file (Chapter 13).
`python scripts/summarize.py`	Table of all `results/full/*_summary.json`.
`python scripts/summarize.py results/lab`	The same, for the lab's results.

Appendix B · Glossary

Adapter (LoRA adapter)
The small set of extra weights produced by LoRA fine-tuning; added to a base model to turn it into the fine-tuned one. MMedPO releases one per dataset (709 MB each).

Parallel runs
Running the same program on different data at the same time. We did it by hand with `&` and `wait` (Slurm also has "job arrays" for this, which we did not need).

Batch job
A script submitted with `sbatch` that runs unattended and writes to a log file.

Batch size
How many inputs a model processes in one call. Ours was 1.

bf16 / fp16 / fp32
Number formats with 2, 2 and 4 bytes per number. Inference used fp16.

Checkpoint
A saved set of model weights (with config files) from which a model can be loaded.

Compute capability / sm_XX
NVIDIA's version number for a GPU's instruction set. B200 = 10.0 = sm_100.

Compute node / login node
The machine that runs jobs (hgx-01) / the machine you log into (aihub-cluster1).

CUDA
NVIDIA's platform and libraries for general-purpose GPU computing.

`CUDA_VISIBLE_DEVICES`
Environment variable that limits which GPUs a program sees and renumbers them from 0.

Decode / prefill
Generating answer tokens one at a time / reading the whole input in one pass.

Driver
System software that lets programs use the GPU. Ours is 615.71.09 (CUDA up to 13.4).

End-to-end time
Per-sample time from opening the image to having the answer text.

Environment variable
A named setting (`NAME=value`) that programs read at start-up, set with `export` .

GRES
Slurm's "generic resources", normally used to hand out GPUs. Not configured on our cluster.

Hugging Face Hub
Website hosting model and dataset repositories; the source of our weights and test sets.

Idempotent
Safe to run more than once with the same result.

Inference / training
Using fixed weights to produce answers / adjusting weights from data.

JSON Lines (.jsonl)
Text file with one JSON object per line; our question and answer format.

Kernel (GPU)
A compiled program that runs on the GPU, such as a matrix multiplication.

KV cache
The model's stored intermediate results for tokens already processed, so they are not recomputed for each new token.

Latency / throughput
Time for one request / requests completed per unit of time.

Merge (LoRA)
Adding the adapter's B·A product into the base weights once, so the result runs like an ordinary model.

NFS
Network file system. `/home` is on NFS, which is why both machines see the same files.

Partition
A Slurm queue of nodes with shared rules. Ours is `hgx` (6-hour maximum).

Percentile (p90)
The value below which 90% of measurements fall.

Pipe (`|`) and `pipefail`
Passing one command's output to the next / a bash option that makes a pipeline fail if any part fails.

PyTorch
The deep-learning library that runs the model's maths on the GPU.

safetensors
A file format for storing model weights safely and efficiently.

Sampling / temperature
Choosing each next token at random according to the model's probabilities / how sharply those probabilities are focused (0.2 = very focused).

Shard
One of several files a large model's weights are split into.

Slurm
The cluster's job scheduler.

Smoke test
A tiny trial run to catch obvious errors before a long run.

Streamer
A Hugging Face hook called as tokens are generated; we used one as a stopwatch.

Symbolic link (symlink)
A file-system shortcut pointing to another file or folder.

Token
A word piece (or an image patch, for the 576 image tokens) that the model reads or writes.

TTFT
Time to first token. About 0.035 s here.

uv
The fast Python package and environment manager we used.

Virtual environment (venv)
A folder with its own Python and packages; ours is `~/MRI-MMedPO/.venv`.

VRAM / GPU memory
The GPU's own memory, where model weights must fit. 183,359 MiB per B200.

Warm-up
The first samples of a run, slower because of one-time set-up costs; excluded from our averages.